

User Manual of DNN+NeuroSim Framework V1.5

Developers: Xiaochen Peng, Shanshi Huang, Anni Lu, Junmo Lee, and James Read, Ming-Yen Lee

PI: Prof. Shimeng Yu, Georgia Institute of Technology

May 1st, 2025

Index

1.	Introduction	1
2.	New Feature Highlights.....	2
2.1	Version 1.4	2
2.2	Version 1.5.3	
3.	System Requirements (Linux)	5
4.	Installation and Usage (Linux)	6
5.	Chip Level Architectures	6
5.1	Interconnect: H-Tree	7
5.2	Floorplan of Neural Networks	8
5.3	Weight Mapping Methods	9
5.4	Pipeline System	11
6.	Circuit Level: Synaptic Array Architectures	12
6.1	Parallel Synaptic Array Architectures	12
6.2	Array Peripheral Circuits	Error! Bookmark not defined.
7.	Device Level: Technology File	25
8.	Algorithm Level: PyTorch Wrapper	27
9.	Algorithm Level: Inference Accuracy Estimation	28
10.	How to run <i>DNN + NeuroSim</i>	34
11.	Reference	38

1. Introduction

DNN+NeuroSim is an integrated framework, which is developed in C++ and wrapped by Pytorch, to emulate the deep neural networks (DNN) inference performance (in V1.0-V1.4) or on-chip training (in

V2.0-V2.1) performance on the hardware accelerator based on near-memory computing or in-memory computing architectures. Various device technologies are supported, including SRAM, emerging non-volatile memory (eNVM) based on resistance switching (e.g. RRAM, PCM, STT-MRAM), and ferroelectric FET (FeFET). SRAM is by nature 1-bit per cell, eNVMs and FeFET in this simulator could support either 1-bit or multi-bit per cell. *NeuroSim* [1] is a circuit-level macro model for benchmarking neuro-inspired architectures (including memory array, peripheral logic, and interconnect routing) in terms of circuit-level performance metrics, such as chip area, latency, dynamic energy and leakage power. With Pytorch wrapper, *DNN + NeuroSim* framework can support hierarchical organization from the device level (transistors from 130 nm down to 7 nm, eNVM and FeFET device properties) to the circuit level (periphery circuit modules such as analog-to-digital converters, ADCs), to chip level (tiles of processing-elements built up by multiple sub-arrays, and global interconnect and buffer) and then to the algorithm level (different neural network topologies), enabling instruction-accurate evaluation on the inference accuracy as well as the circuit-level performance metrics at the run-time of inference.

The target users for this simulator are circuit/architecture designers who wish to quickly estimate the system-level performance with different network and hardware configurations (e.g. device technology choices, sequential read-out or parallel read-out, etc.). Different from our earlier released simulators (*MLP+NeuroSim* [2]), where the network was fixed to a 2-layer MLP and executed purely in C++ (consumes long run-time), this *DNN+NeuroSim* framework is an integrated simulator with Pytorch wrapper (i.e. C++ wrapped by python). With the wrapper, users can define various network structures, precisions of synaptic weights and neural activations, which guarantee efficient inference running with the popular machine learning platforms. Meanwhile, the wrapper will automatically save the real traces (synaptic weights and neural activations) during inference and send them to *NeuroSim* for real-time and real-traced hardware estimation. In this released version, three networks (VGG-8 network for CIFAR-10 dataset, DenseNet-40 network for CIFAR-10 dataset, ResNet-18 network for ImageNet dataset) are provided as default models in the wrapper, with 8-bit synaptic weights and neural activations, while users could modify the precisions and neural network topologies. The hardware parameters (such as technology nodes, memory cell properties, operation modes, and so on) will be defined under *NeuroSim* in **Param.cpp**.

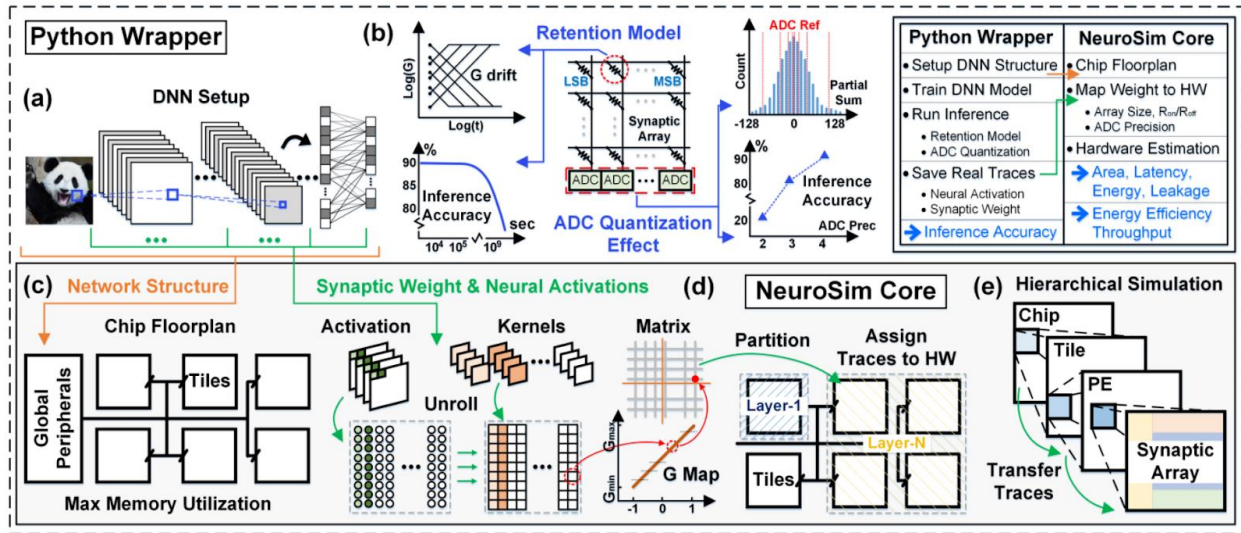


Fig. 1. Framework structure of DNN+NeuroSim. (a) DNN setup in python wrapper, software training with hardware constraints such as weight precision and partial sum quantization; (b) introduction of retention model and ADC

quantization effects to inference accuracy; (c) pre-defined network structure is loaded as input to NeuroSim core, for automatic floor planning which weight-duplication to maximize memory utilization; (d) loading real trace (synaptic weights and neural activations) into NeuroSim, mapping data to conductance and digital voltage input cycles, which are to be partitioned and assigned to different locations of the CIM system; (e) hierarchical simulation from chip to tile, and from processing element (PE) to synaptic array.

2. New Feature Highlights

2.1 Version 1.4

Key changes in this released V1.4 are summarized as follows.

2.1.1 Technology update down to 1 nm node

In NeuroSim 1.4, the technology node for the main array (SRAM, RRAM, etc.) and peripheral circuit design (switchmatrix, interconnect, ADC) are supported down to 1 nm node. The device/standard cell/interconnect parameters are updated to capture the industry trend and IRDS 2021-2022 projections.

2.1.2 Partial parallel mode support

Another key feature of NeuroSim 1.4 is the introduction of partial parallel mode for both PyTorch wrapper and hardware estimation. In previous NeuroSim versions, only fully parallel mode and sequential mode were supported. In practical CIM chip operations, fully parallel MAC operations can result in computation errors caused by various factors. These include sensing errors arising from limited ADC precision, high SNR (Signal-to-Noise Ratio), non-linearity of the MAC output, and the limited on/off ratio of memory cells. These errors can accumulate throughout the neural network layers, leading to a degradation of inference accuracy. To improve the inference accuracy, rows are allowed to be partially enabled and the PPA estimation equations are modified accordingly.

At this time, we only support partial parallel mode for single-level memory cells (i.e. 1b/cell) in the python wrapper. Partial parallel mode is fully supported in the hardware estimations if the users wish to estimate the hardware performance for multi-level cells without including the inference accuracy. We plan to fully support partial parallel mode for multi-level cells in the python wrapper in the next update.

2.1.3 Modification of PPA estimation for few circuit modules

To improve generalizability of the PPA estimation for various subarray sizes and ADC settings, PPA estimation code for flash ADC has been updated from the previous version (V1.3). Detailed explanation on the changes can be found in Section 6.2.

2.2 Version 1.5

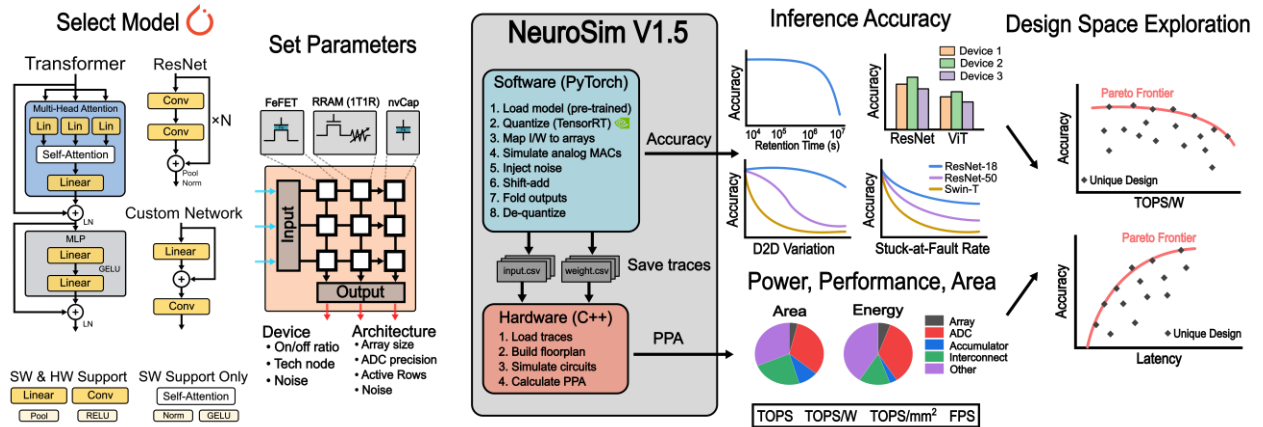


Fig. 2. Overview of NeuroSim V1.5: This update revamped the software backbone to support arbitrary neural network models and provide a more intuitive noise simulation workflow tailored to device and circuit experts. NeuroSim V1.5 enables users to evaluate inference accuracy across diverse architectures, including custom convolutional neural networks and transformers, while simulating various device types and noise sources. The platform uniquely supports power, performance, and area (PPA) estimations, facilitating comprehensive system-level design space exploration. Key new features include optimized noise modeling, compatibility with advanced non-volatile memory technologies, and improved runtime performance.

NeuroSim V1.5 introduces several enhancements, focusing on improved usability, expanded capabilities, and increased performance for benchmarking Compute-in-Memory (CIM) accelerators. Key new features include:

1. TensorRT Integration for Quantization:

- NeuroSim now seamlessly integrates with NVIDIA's TensorRT post-training quantization flow.
- This replaces the previous **WAGE quantization method**, allowing users to evaluate arbitrary pre-trained models (including transformers) without needing retraining.
- Supports automatic operator replacement and configurable bit-widths for inputs and weights.
- Utilizes TensorRT's max or histogram calibration methods for optimal scaling factors.

2. Flexible Noise Modeling Framework:

- Introduces a revamped, flexible noise injection methodology based on pre-characterized statistical models derived from SPICE simulations or silicon measurements.
- Offers two distinct modes:
 - **Device Expert Mode:** Models device-level physics like D2D variation, stuck-at-faults (SAF), and temporal **state drift** using direct device measurements or parameters specified in `mem_states.csv`.
 - **Circuit Expert Mode:** Uses statistical distributions (mean, standard deviation) of MAC outputs derived from SPICE Monte Carlo simulations or tapeout measurements, provided via `output_noise.csv`, to capture aggregated non-idealities.

- Allows for rapid evaluation of diverse device and circuit designs by separating detailed simulation/measurement from the main inference simulation.

3. Expanded Device Support (nvCap):

- Adds support for emerging non-volatile capacitive (nvCap) memories operating in the charge domain.
- Includes new hardware models in the C++ PPA estimator for charge-based computation circuits (e.g., nvCap arrays, charge amplifiers).
- Leverages insights from recent research on nvCap benefits (e.g., improved energy/area efficiency, good scalability, negligible static power/disturbance) and variation analysis.

4. Enhanced Transformer Model Support:

- Provides specific methodology for evaluating transformer models (e.g., Swin Transformer).
- Linear layers within transformers are mapped to CIM arrays for acceleration.
- Dynamic operations (self-attention, normalization, activation functions) are assumed to be handled by standard digital/floating-point units, acknowledging the limitations of NVM-based CIM for frequent reprogramming. *(Note: Current PPA estimation excludes these digital/FP units).*

5. Significantly Improved Runtime Performance:

- Achieves up to 6.5x faster behavioral simulation runtime compared to NeuroSim V1.4.
- Optimizations stem from leveraging GPU-accelerated PyTorch tensor operations for core computations and using efficient statistical noise modeling instead of direct simulation within the inference loop.

These updates make NeuroSim V1.5 a more powerful and efficient tool for exploring the complex design space of CIM accelerators, bridging detailed device/circuit analysis with system-level accuracy and hardware performance evaluation.

3. System Requirements (Linux)

The tool is expected to run in Linux with required system dependencies installed. These include GCC, GNU make, GNU C libraries (glibc). We have tested the compatibility of the tool with a few different Linux environments, such as (1) Red Hat 7.8 (Maipo), gcc v4.8.5, glibc v2.17, (2) Ubuntu 16.04, gcc v5.5.0, glibc v2.23, and they are all workable.

※ The tool may not run correctly (stuck forever) if compiled with gcc 4.5 or below, because some C++11 features are not well supported.

4. Installation and Usage (Linux + Anaconda)

See the README in the respective branch of the main NeuroSim repo for the version you are installing. For DNN+NeuroSim V1.5 go to https://github.gatech.edu/Yu-Lab/DNN_NeuroSim_V1.5

5. Chip Level Architectures

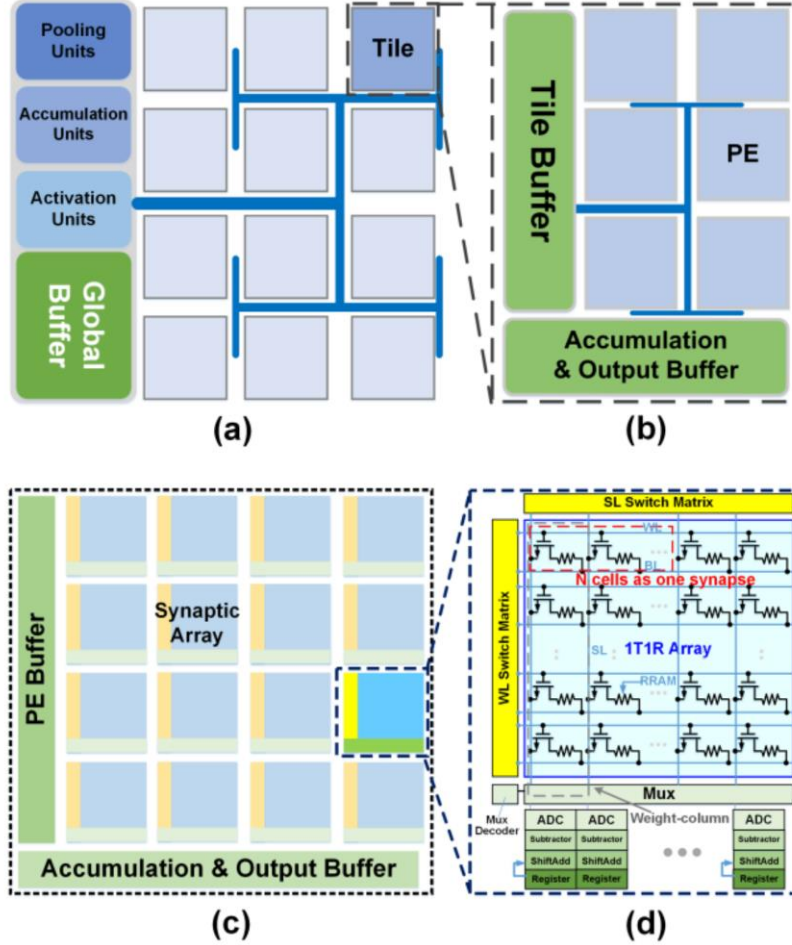


Fig. 3. The diagram of (a) top level of chip architecture, which contains multiple tiles, global buffer, accumulation units, activation units (sigmoid or ReLU) and pooling units; (b) a tile with multiple processing elements (PEs), tile buffer to load in activations, accumulation modules to add up partial sums from PEs and output buffer; (c) a PE contains a group of synaptic arrays, PE buffer and control units, accumulation modules and output buffer; (d) an example of synaptic array based on one-transistor-one-resistor (1T1R) architecture.

In this framework, we assume the on-chip memory is sufficient to store synaptic weights of the entire neural network, thus the only off-chip memory access is to fetch in the input data. Fig. 3 shows the modeled chip hierarchy, where the top level of chip consists of multiple tiles, global buffer, accumulation units, activation units (sigmoid or ReLU), and pooling units. Fig. 3 (b) shows the structure of a tile, which contains several processing elements (PEs), tile buffer to load in neural activations, accumulation modules to add up partial sums from PEs and output buffer. Similarly, as Fig. 3 (c) shows, a PE is built up by groups of synaptic sub-arrays, PE buffers, accumulation modules and output buffer. Fig. 3 (d) shows an example of synaptic sub-array, which is based on one-transistor-one-resistor (1T1R) architecture for

eNVMS. At sub-array level, the array architecture is different for SRAM or FeFET (not shown in this figure).

5.1 Interconnect: H-Tree

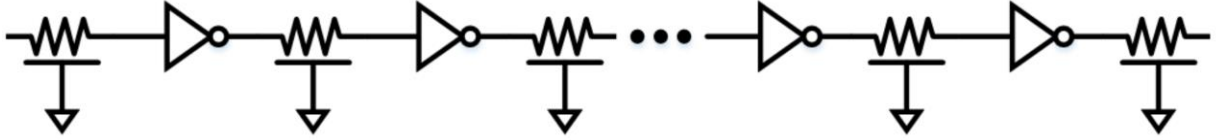


Fig. 4. Diagram of a wire with repeaters

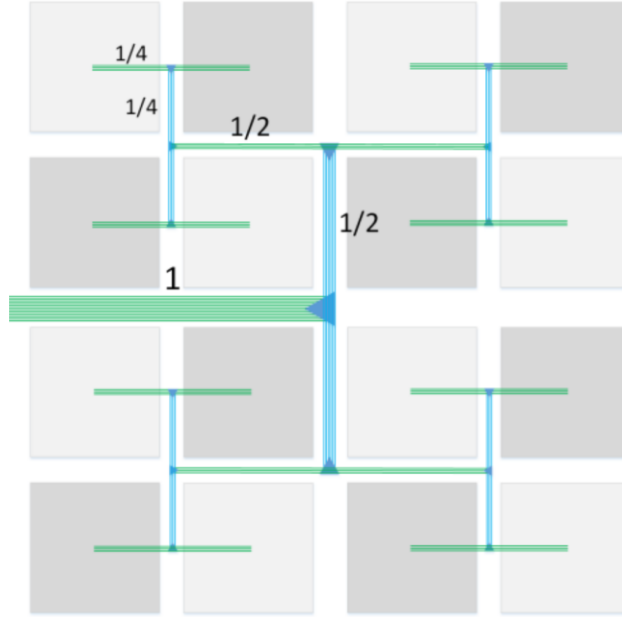


Fig. 5. An example of the H-tree interconnect for a 4×4 computation-unit array

To estimate the area, latency, dynamic energy and leakage of the interconnect, we assume the routing among modules in each hierarchy is based on an H-tree structure. According to interconnect engineering, the wire delay could be reduced by introducing repeaters which is used to split the wire into multiple segments. As Fig. 4 shows, a wire could be considered as a group of wire segments and repeaters. To find the optimal length of wire segment between repeaters leading to minimum delay, a VLSI design function [5] is introduced in EQ (4.1). R is the resistance of a minimum-sized repeater, C is the gate capacitance, $C_{p_{inv}}$ is the diffusion capacitance, R_w and C_w are the unit resistance and capacitance of wire, respectively.

$$L_{optimal} = \sqrt{\frac{2RC(1+p_{inv})}{R_w C_w}} \quad (4.1)$$

The repeater size should use an NMOS transistor width of

$$W = \sqrt{\frac{RC_w}{R_w C}} \quad (4.2)$$

Alternatively, to limit the energy consumption of interconnect, we may find a semi-optimal design option of trade-offs between wire latency and energy. In this framework, we introduce two parameters called “globalBusDelayTolerance” (and “localBusDelayTolerance” for global bus and tile/PE local bus

respectively) to find the semi-optimal floorplan of bus with such delay sacrifice. These parameters are defined in **param.cpp**.

Fig. 5 shows an example of H-tree structure for 4×4 computation units (either tiles or PEs), where the bus width connected to each unit is assumed to be same. The H-tree is composed of multiple stages (horizontal and vertical) from the widest (main bus) to the narrowest (connected to computation units). The wire length decreases by $\times 2$ at each stage from wide to narrow ones, while the sum of bus width at each stage is fixed to the width of main bus. We find that when there are many computation units, the H-tree structure becomes inefficient with long detours and limited bus width for each unit. Therefore, we support horizontal x bus and vertical y bus to connect between global buffer and tiles in the 1.4 version. To enable the X-Y bus interconnect, set “globalBusType = false” in **Param.cpp**.

5.2 Floorplan of Neural Networks

To map various neural networks according to the defined chip architecture, it is crucial to follow a certain rule which does not violate hardware structure (and data flow) while guarantees high-enough memory utilization. We defined an algorithm to automatically generate the floorplan based on two kinds of weight-mapping methods, which optimize the memory utilization and define the tile size, PE size, number of tiles needed, based on user-defined synaptic array size.

The floorplan starts from tile sizing to PE sizing, while the size of synaptic array is defined by users in **Param.cpp**. With pre-defined network structure and weight mapping method, *NeuroSim* automatically calculate weight-matrix size for each layer (especially for convolutional ones, where 3D kernels will be unrolled to 2D matrices), the tile size firstly is set to a maximum value which could contain the largest weight-matrix among all the layers, then *NeuroSim* calculate the memory utilization (defined as memory mapped by synaptic weights / total memory storage on chip), keep decreasing the tile size till *NeuroSim* find a solution with optimal memory utilization.

To further increase memory utilization and speed up the processing speed of whole network as much as possible, weight duplication is introduced to each layer. Since the layer structure (such as input feature size, channel depth and kernel size) varies significantly in DNNs, which could occupy various amounts of synaptic arrays, it is possible that, the weight of several layers cannot fully fill one PE or even one synaptic array, a naïve way to custom-design the hardware is to mix multiple such small layers into one tile (or even one PE), however, this could make it complicated to define tile/PE size and number of tiles needed, thus, in this framework, we assume one tile is the minimum computation units for each layer, i.e., it is not allowed to map more than one layer into one tile, but there could be multiple tiles to map one single layer.

Hence, similarly, *NeuroSim* will continue to decide the PE size and possibilities of weight duplication among PEs, with pre-defined tile size as discussed above. For example, if the weight-matrix of a specific layer is smaller than the tile size (which means the tile cannot be fully filled by one weight-matrix), it is possible to duplicate the weight-matrix and fetch in multiple neural activation vectors, thus to speed up the process of this layer. In this step, *NeuroSim* start the PE design with a maximum PE size which equals to half of the tile size (to guarantee the exist of defined hierarchy), and decide whether to duplicate the weight-matrix and how many times of duplication for each layer, then recalculate the memory utilization with weight duplication factors, keep decreasing the PE size till *NeuroSim* find the optimal solution with highest memory utilization.

Finally, weight duplication could be further utilized inside PE, i.e. duplicate weight among synaptic arrays, in the similar way as PE design, the only difference is the synaptic array size if fixed. With these

three stage floorplans, *NeuroSim* could guarantee high-enough memory utilization, meanwhile optimize the inference process speed.

Table I shows the overall memory utilization of the floorplan algorithm of AlexNet, VGG-16 and ResNet-34, based on the two supported mapping methods for ImageNet dataset, as well as the VGG-8 network for CIFAR-10 dataset. The results were based on assumption that one memory cell is sufficient to map one synaptic weight (i.e. an 8-bit cell to map an 8-bit synapse), and synaptic array size is 128×128 . With various hardware configuration (such as two 4-bit memory cells form one 8-bit synaptic weight), the memory utilization could be slightly different.

Table I Memory Utilization

Network	Conventional Mapping	Novel Mapping
VGG-8 (CIFAR-10)	90.65%	98.73%
AlexNet	98%	97%
VGG-16	98.79%	99.24%
ResNet-34	85.88%	90.13%

5.3 Weight Mapping Methods

We support two mapping methods in this framework, conventional mapping and novel mapping method which was proposed in [6]. Fig. 6 shows the example of conventional mapping for one convolutional layer, where each 3D kernel (weight) is unrolled into a long column, since the partial sums in each 3D will be summed up to get the final output. Thus, the total kernels in each convolutional layer will form a group of such long columns, i.e., a large weight matrix.

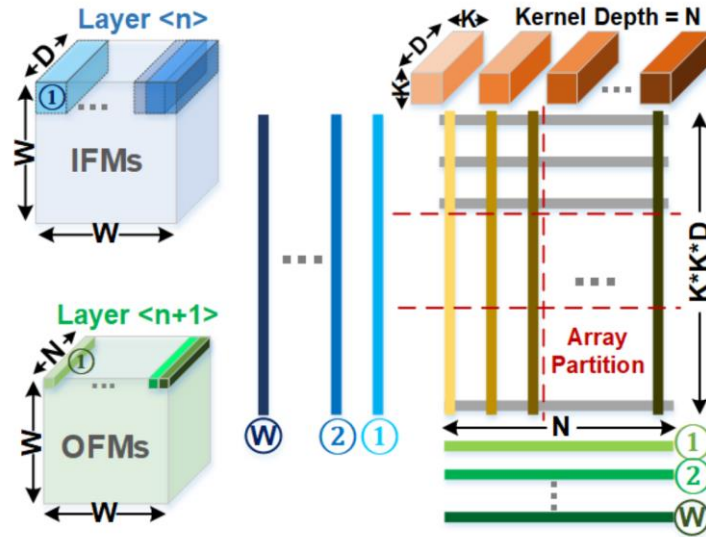


Fig. 6. An example of conventional mapping method of input and weight data.

To get the output feature maps (OFMs), as Fig. 6 shows, at first cycle, a part of input feature maps (IFMs) (shown in dark blue cube) will be multiplied with each 3D kernels. If we assume a single OFM has size of $W \times W$, with channel depth of N , there are N such OFM in total, we call the front OFM as the first OFM, and the back one as the N^{th} OFM. In this way, the sum of dot-products from the first kernel will be the

first element in the first OFM, the sum of dot-products from the second kernel will be the first element in the second OFM, and so on, thus, at the first cycle, we could get the first elements in every OFM from front to back (as shown in light green row in size $1 \times 1 \times N$). In the same way, at the second cycle, the kernels will “slide over” the inputs with a stride (equals to one in this example), after the dot-product operation, we will get all the second elements in each OFM. Thus, to generate the total OFMs in layer $\langle n \rangle$, we need to “slide over” the IFMs by $W \times W$ times, i.e. we need $W \times W$ cycles to finish the computation.

It should be noted that, in conventional mapping, during the entire operation, a part of the IFMs used in earlier cycle will always be reused at current cycle. Considering about the huge amount of dot-product operations in convolutional layers, these frequent revisiting of input data from upper-level buffers could cause a significant energy and latency waste. Thus, a novel mapping method is introduced to maximize input data reuse.

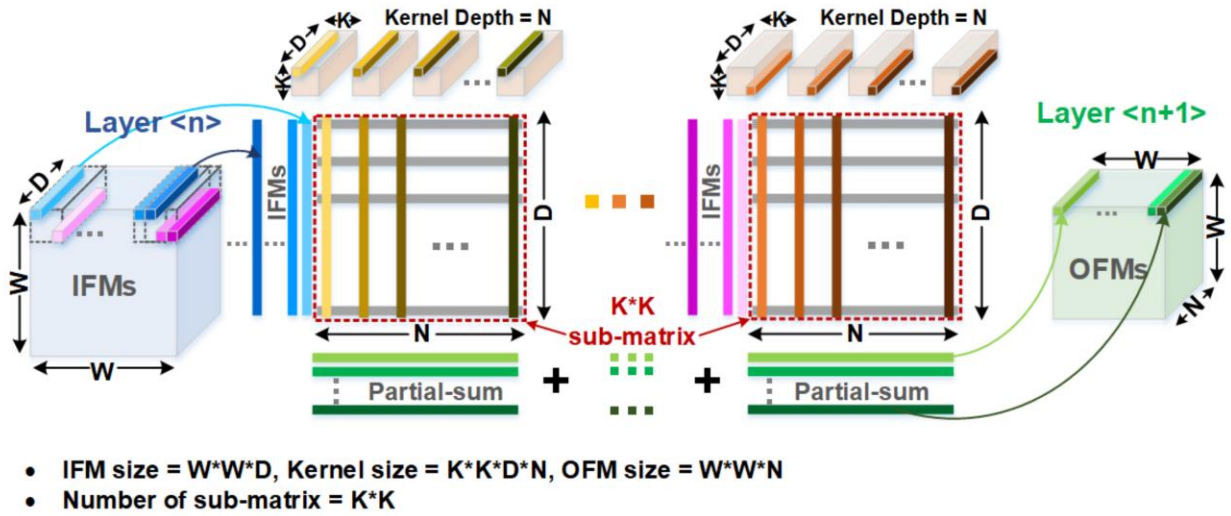


Fig. 7. An example of the novel mapping method of input and weight data.

Fig. 7 shows an example of novel mapping for the same convolutional layer. Instead of unrolling 3D kernels into a large matrix, the weights at different spatial location of each kernel are mapped into different sub-matrices. According to the spatial location of partitioned kernel data in each kernel, we define which group of these partitioned kernel data should belong to. Hence, $K \times K$ sub-matrices are needed for the kernels (whose first and second dimension equal to K and K), since each sub-matrix has size $D \times N$, the size of total weight matrix will be $K \times K \times D \times N$, which equals to the size of unrolled matrix from conventional mapping method (as Fig.6 shows). Similarly, the input data which should be assigned to various spatial location in each kernel, will be sent to the corresponding sub-matrix, respectively. Partial sums from sub-matrices could be obtained in parallel. Later, an adder tree will be used to sum up the partial sums.

Hence, such group of sub-arrays with the necessary input and output buffers and accumulation modules can be defined as a processing element (PE). The kernels are split into several PEs according to their spatial locations, and assign the input data into corresponding ones, it is possible to reuse the input data among these PEs, i.e., directly transfer input data among PEs which do not need to revisit upper-level buffers.

5.4 Pipeline System

In this framework, we assume all the synaptic weights are mapped on to the inference chip, which means it is possible to build up a pipeline system with acceptable global buffer overhead (to save activations for different images), to improve throughput and energy efficiency (less leakage for idle cycles).

To avoid overhead of complicated control circuits, we assume each layer as one pipeline stage, and the pipeline system clock cycle is defined as the longest latency among all the layers. According to the mapping method of synaptic weights, the total latency of each layer is related to the size of its input feature maps (IFMs) and stride size.

Since the IFMs tend to become deeper but smaller from shallow layers to deeper layers, the speed of deeper layers will be limited by the shallow layers, since they have to wait for the shallow layers to generate the IFMs. During the waiting period, the deeper layers have to stay idle, and thus cause leakage energy.

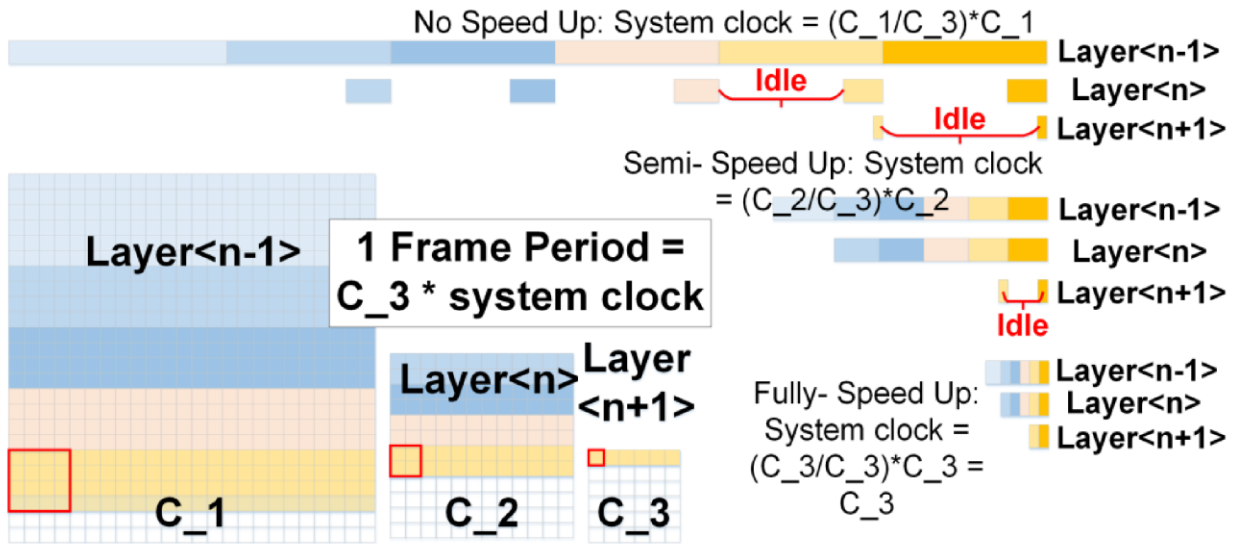


Fig. 8. Speed-up in the pipelined system.

In this case, we defined a parameter “speedUpDegree” in file “Param.cpp” to speed up each layer, by duplicating the weight and processing different IFMs simultaneously. As Fig. 8 shown below, if the size of IFMs is $(C_1 * C_1)$, with 2X speed up, the actual latency will be $(C_1 * C_1)/4$. It should be noted that, in this framework, to avoid ultimate speed-up (i.e. ultimate weight duplication), we define a boundary of speed-up degree as the maximum speed-up allowed: there is no idle period across all the layers after speed-up.

6. Circuit Level: Synaptic Array Architectures

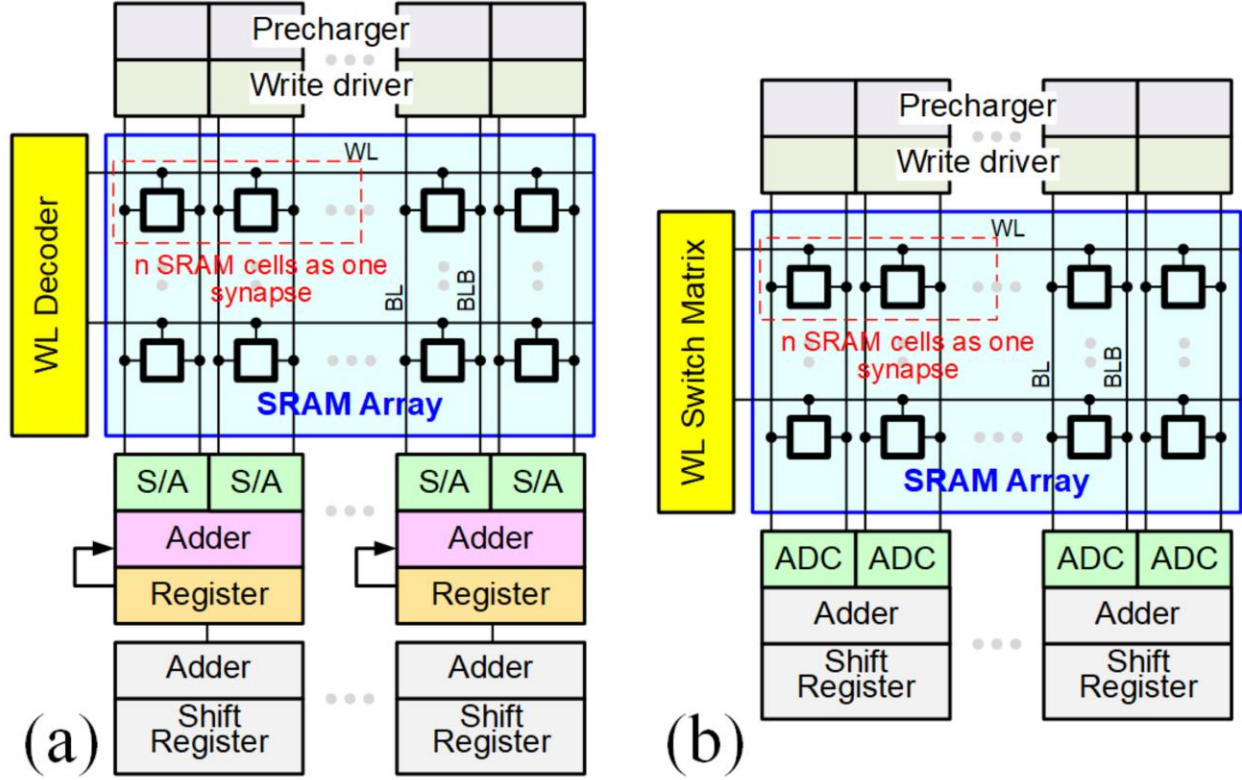


Fig. 9. The diagram of SRAM-based (a) sequential-read-out; (b) parallel-read-out synaptic arrays.

With various device technologies, the chip could operate in different modes, such as digital sequential (row-by-row) read-out for near-memory computing, or analog parallel read-out for in-memory computing. In the simulator, the parameters of synaptic devices and synaptic array modes will be instantiated in `param.cpp`.

6.1 Parallel Synaptic Array Architectures

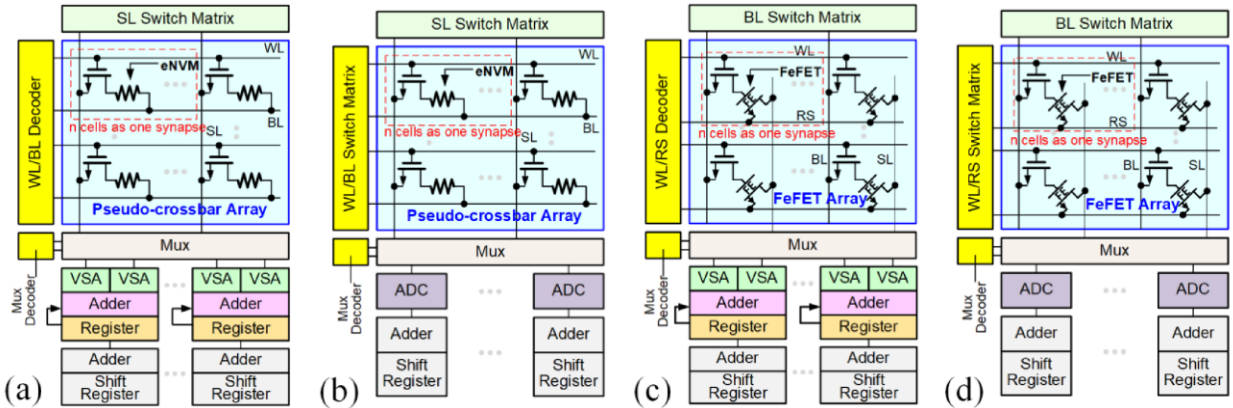


Fig. 10. (a) sequential-read-out and (b) parallel-read-out analog eNVM pseudo-1T1R synaptic arrays; (c) sequential-read-out and (d) parallel-read-out analog FeFET synaptic arrays.

Fig. 9 and Fig. 10 show three kinds of supported synaptic arrays, which could be used to process analog in-memory computing. Here are some assumptions that apply to all kinds of array architectures below. The higher precision than 1-bit in the input neuron activation is represented by multiple cycles of input voltage signals to the row, and no analog voltage is used to represent the input, thus no digital-to-analog converter (DAC) is used, as the nonlinearity in I-V curve of eNVMs will introduce distortion in parallel read-out [7]. The higher precision than 1-bit in the weight could be represented by a single analog synaptic cell or multiple synaptic cells. For example, an 8-bit weight could be represented by a single 8-bit eNVM cell (assuming it is technologically viable), or 2 eNVM cells (4 bits per cell), or 4 eNVM cells (2 bits per cell), or 8 eNVM binary cells. In our design, the inference is performed in parallel mode by activating all the rows, while the weight update in the training is performed in a row-by-row fashion. It should be noted that as the peripheral ADC size is typically much larger than the column pitch of the array, therefore column sharing is used by the column mux (e.g. 8 columns share one ADC). Although the highest parallelism is achieved with all rows activated, the column currents become high with synaptic resistances parallel connected. It causes large IR drop on wires and column mux and high energy consumption. To balance this issue, partial parallel mode is added to the 1.4 version to enable customized number of rows. The leakage of SRAM is included with the rows in idle as well.

6.1.1 SRAM synaptic array

Multiple digital SRAM cells can be grouped along the row to represent one weight with higher precision than 1-bit, as shown in Fig. 9. The weighted sum and weight update operations are similar to the row-by-row read and write operations in conventional SRAM for memory, respectively. In sequential-read-out mode as Fig. 9 (a) shows, to select a row, the WL is activated through the WL decoder. To access all the cells on the selected row, the BLs are pre-charged by the pre-charger and the write driver in weighted sum and weight update, respectively. After the memory data are read by the sense amplifier (S/A), the adder and register are used to accumulate the partial weighted sum in a row-by-row fashion. In parallel-read-out mode as demonstrated in [8], the input vectors will be fetched in via WL switch matrix, the partial-sums will be collected along columns simultaneously at one time with high-precision flash-ADCs based on multilevel S/A by varying references. In both modes, the adders and shift registers are used to shift and accumulate partial sums for multiple cycles of input vectors (which represent MSB to LSB of the analog neural activations).

6.1.2 Analog eNVM 1T1R synaptic array

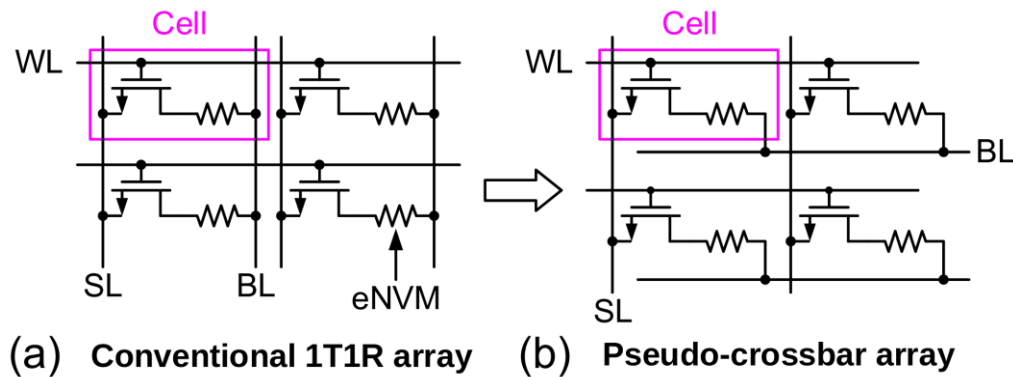


Fig. 11. Transformation from (a) conventional 1T1R array to (b) pseudo-crossbar array by 90° rotation of BL to enable weighted sum operation.

Fig. 10 (a) and (b) shows the structure of 1T1R based eNVM array. The WL controls the gate of the transistor, which can be viewed as a switch for the cell. The source line (SL) connects to the source of the transistor. The eNVM cell's top electrode connects to the BL, while its bottom electrode connects to the drain of the transistor through a contact via. In such case, the cell area of 1T1R array is then determined by the transistor size, which is typically $>6F^2$ depending on the maximum current required to be delivered into the eNVM cell. Larger current needs larger transistor gate width/length (W/L). However, conventional 1T1R array is not able to perform the parallel weighted sum operation. To solve this problem, we modify the conventional 1T1R array by rotating the BLs by 90° , which is known as the pseudo-crossbar array architecture, as shown in Fig. 11 (b). In weighted sum operation, all the transistors will be transparent when all WLs are turned on. Thus, the input vector voltages are provided to the BLs, and the weighted sum currents are read out through SLs in parallel. Then the weighted sum currents are digitalized by a current-mode sense amplifier (S/A), and a Flash-ADC with multilevel S/A by varying references.

6.1.3 Analog eNVM crossbar array

The crossbar array structure has the most compact and simplest array structure for analog eNVM devices to form a weight matrix, where each eNVM device is located at the cross point of a word line (WL) and a bit line (BL), as shown in Fig. 10 (c). The crossbar array structure can achieve a high integration density of $4F^2/\text{cell}$ (F is the lithography feature size). If the input vector is encoded by read voltage signals, the weighted sum operation (matrix-vector multiplication) can be performed in a parallel fashion with the crossbar array. Here, the crossbar array assumes there is an ideal two-terminal selector device connected to each eNVM, which is desired for suppressing the sneak path currents during the row-by-row weight update. It should be noted that ideal selector device is still under research and development.

6.1.4 Analog FeFET array

As shown in Fig. 10 (c) and (d), the analog FeFET array is in the pseudo-crossbar fashion as proposed in [9], which is similar to the analog eNVM pseudo-crossbar one. It also has an access transistor for each cell to prevent programming on other unselected rows during row-by-row weight update. As FeFET is a three-terminal device, it needs two separate input signals to be fetched to activate WLs and introduce read voltages to RS (read select), respectively, where RS is used to fetch in input vectors as Fig. 12 shown below.

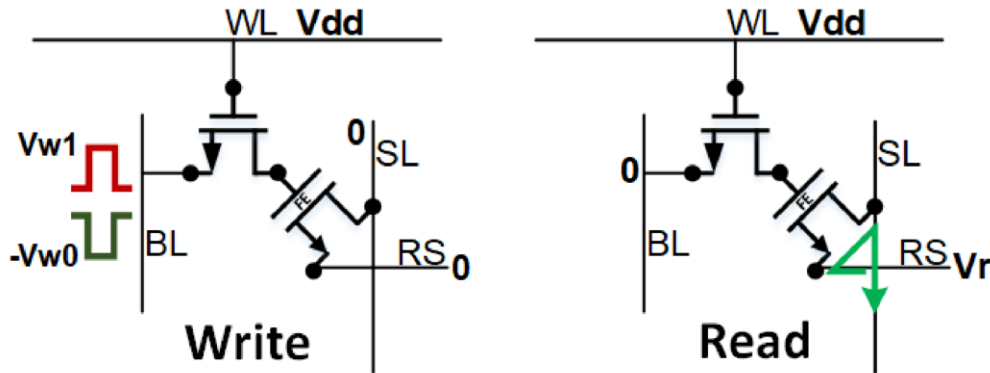


Fig. 12. Operations of (a) write and (b) read in FeFET cell.

6.1.5 Non-Volatile Capacitive (nvCap) Synaptic Array

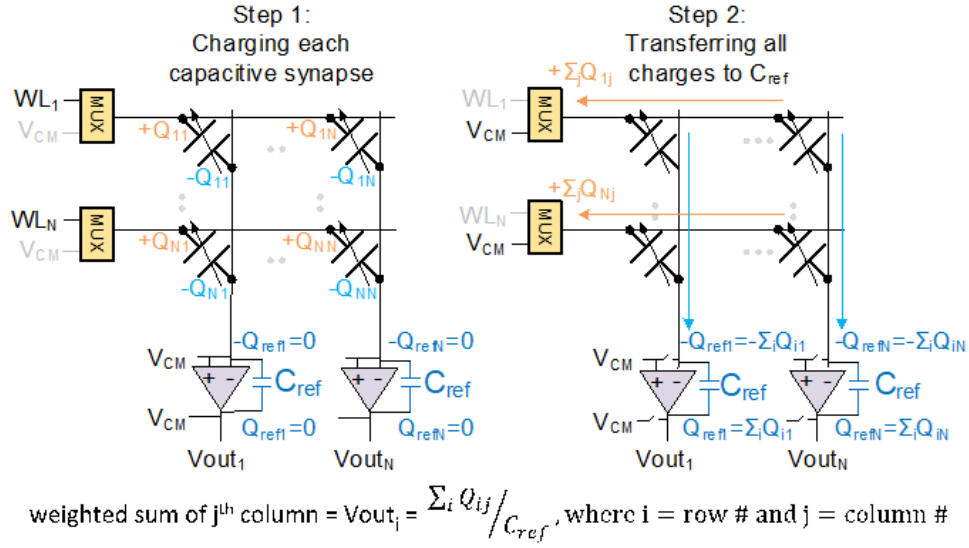


Fig. 13. Operating principle of the non-volatile capacitive CIM. The first step is to charge each non-volatile tunable capacitor with input voltage from the WL, where charges on each capacitor represent the product of each entry. The second step transfers the charges of each column to the reference capacitor (C_{ref}) shunting the OPAMP. The charges result in an output voltage (V_{out}) as the weighted sum.

NeuroSim V1.5 adds support for non-volatile capacitive (nvCap) memory arrays, which operate in the charge domain for compute-in-memory. In this architecture, synaptic weights are stored as programmable capacitance values in nvCap cells (e.g., MFS-type ferroelectric capacitors). MAC operations occur in two phases: first, input voltages charge the capacitors, and second, the stored charge is transferred and summed onto a reference capacitor shunting an operational amplifier, producing an output voltage proportional to the weighted sum. This charge-based approach offers advantages like negligible static power, minimal IR drop allowing better scalability, and reduced read disturbance compared to some resistive memories, eliminating the need for access transistors. NvCap arrays are subject to non-idealities like D2D variation and temporal noise, which impact the trade-offs between accuracy, latency (column delay), and parallelism. NeuroSim V1.5 includes updated hardware models for nvCap PPA estimation and allows users to select this memory type via `memcelltype = 4` in `Param.cpp`. Accuracy under these non-idealities can be evaluated using the python wrapper. See [11]-[12] for a detailed analysis of this new memory device.

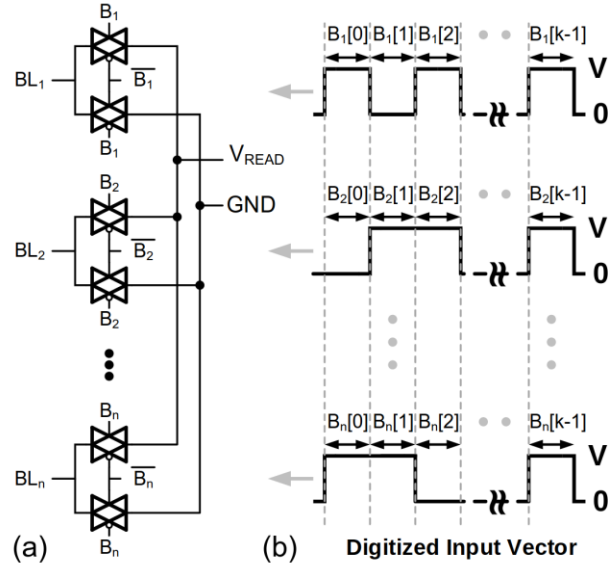


Fig. 15. (a) Transmission gates of the BL switch matrix in the weighted sum operation. A vector of control signals (B_1 to B_n) from the registers (not shown here) connect the BLs either to a voltage source or ground. (b) Control signals in a bit stream to represent the precision of the input vector.

6.2.3 Crossbar WL decoder

The crossbar WL decoder is modified from the traditional WL decoder. It has an additional feature to activate all the WLs for making all the transistors transparent for weighted sum. The crossbar WL decoder is constructed by attaching the follower circuits to every output row of the traditional decoder, as shown in Fig. 16. If $ALLOPEN=1$, the crossbar WL decoder will activate all the WLs no matter what input address is given, otherwise it will function as a traditional WL decoder. In the simulator, the crossbar WL decoder contains a traditional WL decoder (**wlDecoder**) instantiated from **RowDecoder** class in **RowDecoder.cpp** and a collection of follower circuits (**wlDecoderOutput**) instantiated from **WLDecoderOutput** class in **WLDecoderOutput.cpp**, this module is used in sequential-read-out synaptic arrays.

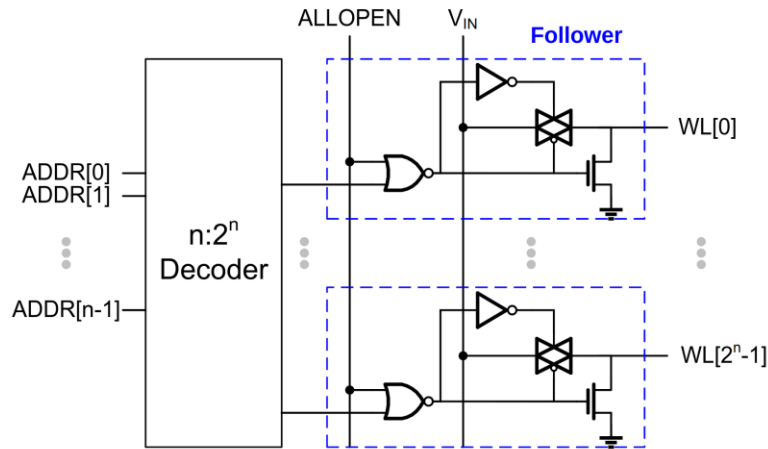


Fig. 16. Circuit diagram of the crossbar WL decoder. Follower circuit is attached to every row of the decoder to enable activation of all WLs when $ALLOPEN=1$.

6.2.4 Decoder driver

The decoder driver helps provide the voltage bias scheme for the write operation when its decoder selects the cells to be programmed. As the digital eNVM crossbar array has the write voltage bias scheme for both WLs and BLs, it needs the WL decoder driver (**wlDecoderDriver**) and column decoder driver (**colDecoderDriver**). These decoder drivers can be instantiated from **DecoderDriver** class in **DecoderDriver.cpp**, this module is used in sequential-read-out synaptic arrays.

6.2.5 New Decoder Driver and Switch Matrix

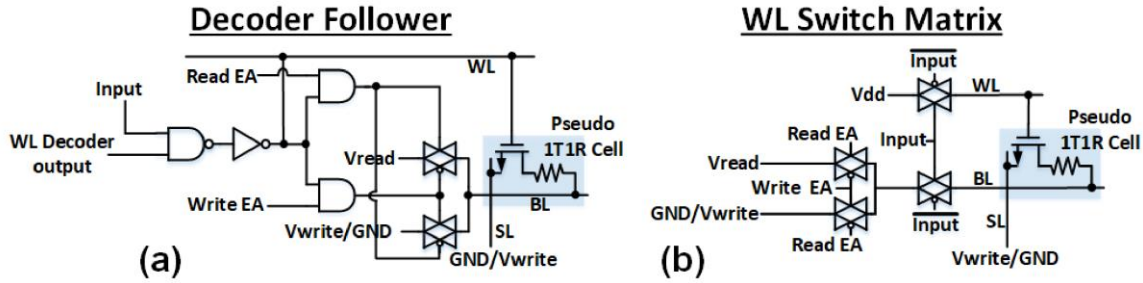


Fig. 17. Circuit diagram of (a) decoder follower and (b) WL switch matrix, which are used to control both WLs and BLs simultaneously, for pseudo-1T1R synaptic arrays.

One should be noticed that, for eNVM pseudo-crossbar and FeFET synaptic arrays, the WLs and BLs/RSs could be controlled by same input signals, but with different voltage values, thus, it could significantly save the area for unnecessary BL/RS switch matrix. To achieve this function, there are several extra control gates to be added into the WL decoder driver circuits, and into the WL switch matrix. Fig. 17 shows the circuit diagram of new decoder driver and switch matrix for eNVM pseudo-1T1R synaptic array, which could be used to control both WL and BL (or RS) at the same time. In Fig. 17 (a), with the input and decoder output, both of WL and BL will be controlled, where the WLs will be either activated or not, and the BLs to be connected to either the read voltage or ground. Similarly, in Fig. 17 (b), the each single WL switch matrix has two extra transmission gates to be used to send two separate voltages into the corresponding WL and BL. In FeFET synaptic arrays, the signals connected to BLs in this example, will be connected to RSs. In the simulator, the WLNewDecoderDriver (decoder driver) is instantiated from **WLNewDecoderDriver** class in **NewDecoderDriver.cpp** and the WLNewSwitchMatrix (WL switch matrix) is instantiated from **WLNewSwitchMatrix** class in **NewSwitchMatrix.cpp**, these new decoder follower and switch matrix are used in eNVM pseudo-1T1R and FeFET synaptic arrays.

6.2.6 Multiplexer (Mux) and Mux decoder

The Multiplexer (Mux) is used for sharing the read periphery circuits among synaptic array columns, because the array cell size is much smaller than the size of read periphery circuits and it will not be area-efficient to put all the read periphery circuits underneath the array. However, sharing the read periphery circuits among synaptic array columns inevitably increases the latency of weighted sum as time multiplexing is needed, which is controlled by the Mux decoder. In the simulator, the Mux (**mux**) is instantiated from **Mux** class in **Mux.cpp** and the Mux decoder (**muxDecoder**) is instantiated from **RowDecoder** class in **RowDecoder.cpp**.

6.2.7 Analog-to-digital converter (ADC)

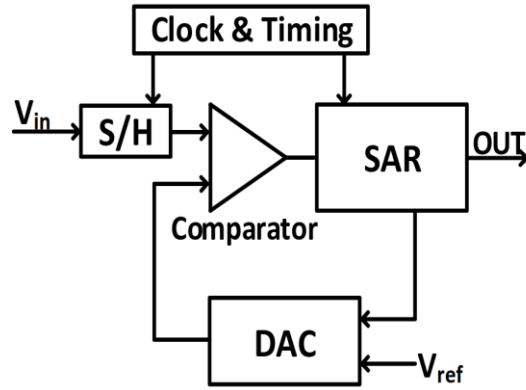
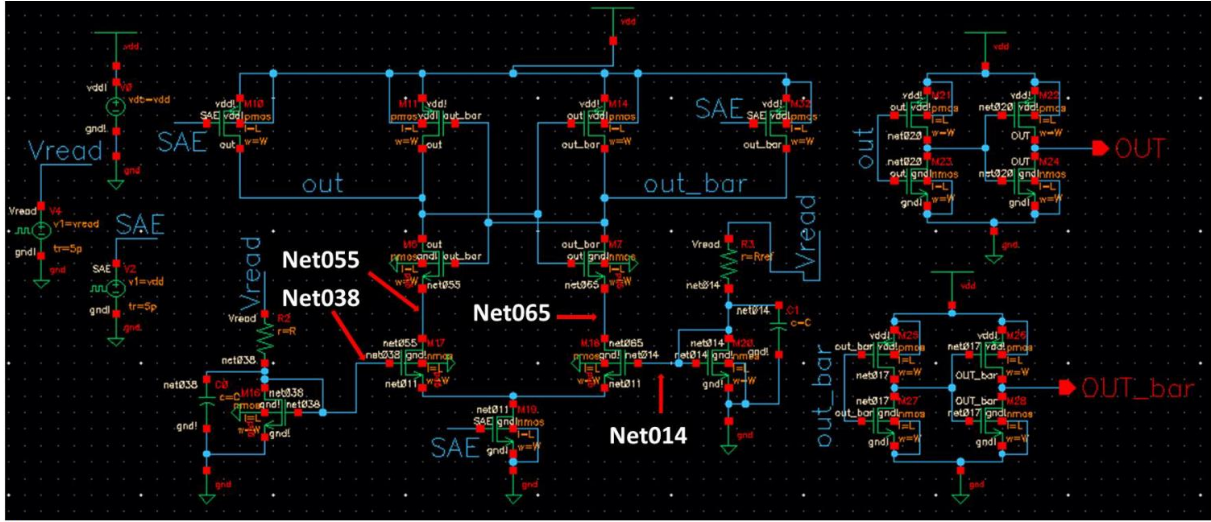


Fig. 18. Schematic of successive-approximation-register (SAR) ADC.

To read out the partial-sums and further process them in the subsequent logic modules (such as activation and pooling), ADCs are used at the end of SLs to generate digital outputs. In the simulator, different types of ADC are supported such as successive-approximation-register (SAR) ADC (Fig. 18), and Flash ADC using multilevel voltage-mode sense amplifiers (VSA) or current-mode sense amplifier (CSA) (Fig. 19). The flash ADC design/modeling for NeuroSim 1.4 has been updated from the previous version, to improve the generalizability to different synapse types (RRAM synapse, SRAM synapse) and the flexibility of supporting PPA estimation for various subarray sizes/ADC precisions. Additionally, new sets of flash ADC parameters (for power/latency/area estimation) are incorporated into the code based on the cadence simulation results on the updated ADC circuits (Fig. 20).

A PTM model is used for the device compact model for Cadence simulation. For the technology nodes beyond 14 nm, calibrated PTM models (introduced in Section. 7) to match the on/off current & gate capacitance requirement at each technology node were used. We point out general working principles of ADC circuit operation before detailing the modeling process for ADC in NeuroSim 1.4. In theory (and from the cadence simulation), the ADC latency can have a wide variety of values ranging from the ps level to the tenths of ns level depending on the read voltage (V_{read}) of the main array and the closeness between the column resistance and the reference resistance. Generally, the higher the read voltage, the faster development of the voltage difference between the two storage nodes in the latch (part of sense amplifier), which leads to faster ADC sensing. Also, the higher the reference resistance and column resistance difference, the faster the sensing time. (In the case of SRAM mode, the supply voltages for the latch in the SRAM are assumed to be adjusted to V_{read} during ADC operation, for consistent modeling with the RRAM mode). As a side effect of increasing read voltage, the dynamic energy consumption of the bitline increases due to the increased read voltage, leading to higher energy consumption in ADC, as the dynamic energy consumption is proportional to $C_{\text{bit}}V_{\text{dd}}^2$ (C_{bit} = bitline (column) capacitance).

(a)



(b)

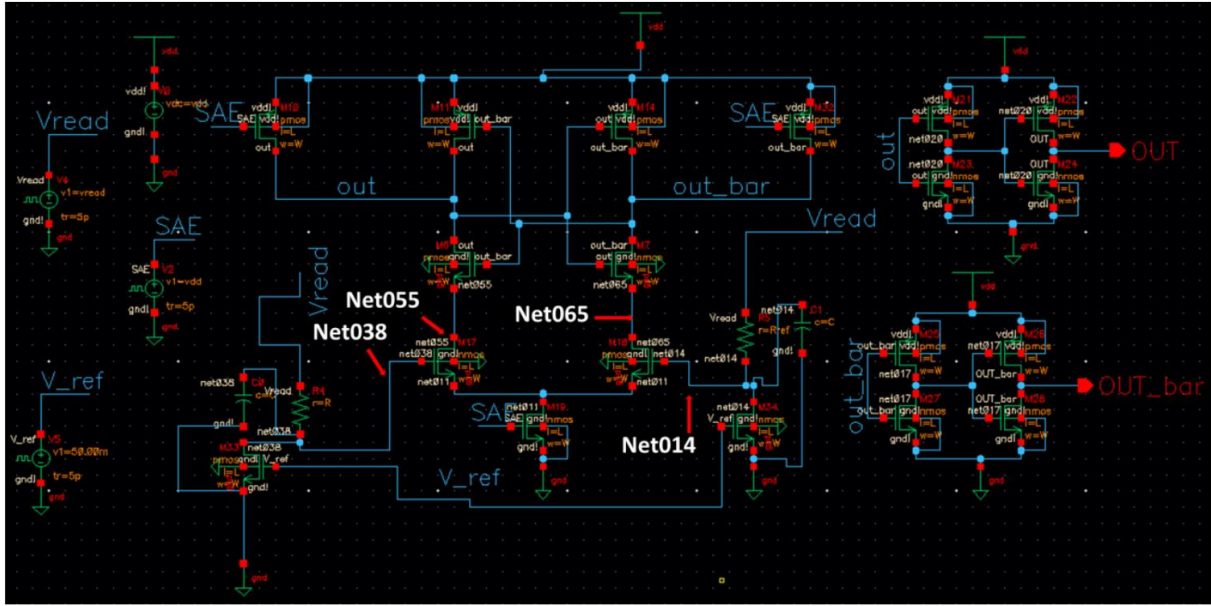


Fig. 19. Schematic of (a) current sense amplifier and (b) voltage sense amplifier.

Due to the complexity of modeling the latency for arbitrary read voltage values, we set a fixed read voltage for each technology node based on a certain criterion. For the default implementation provided in NeuroSim 1.4 public version, the read voltage is chosen to satisfy following requirements: (i) ~ 1 ns of 5-bit flash ADC latency is achieved at 22 nm node and (ii) 5-bit ADC latency decreases by 100~300 ps with each successive technology node scaling. The details of each power/performance/area estimation are elaborated in the following paragraphs.

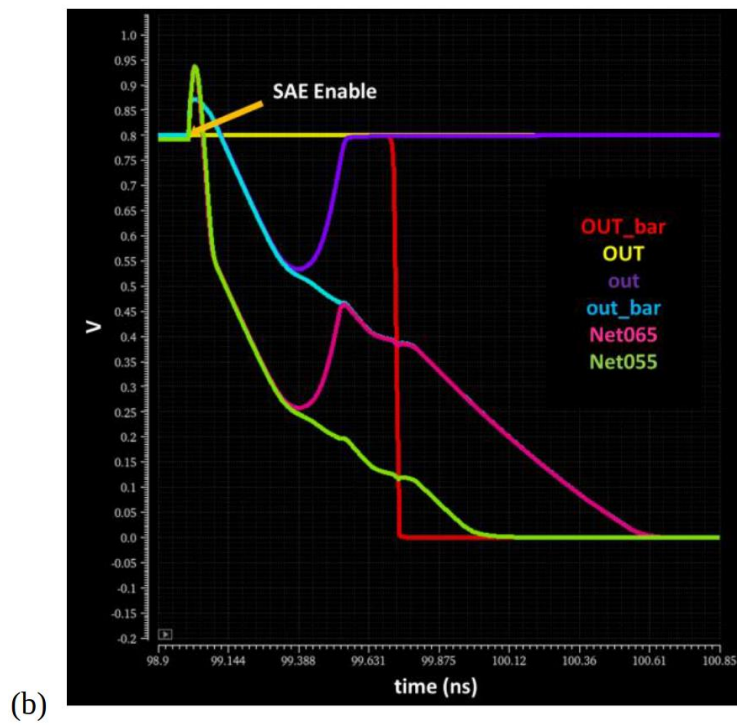
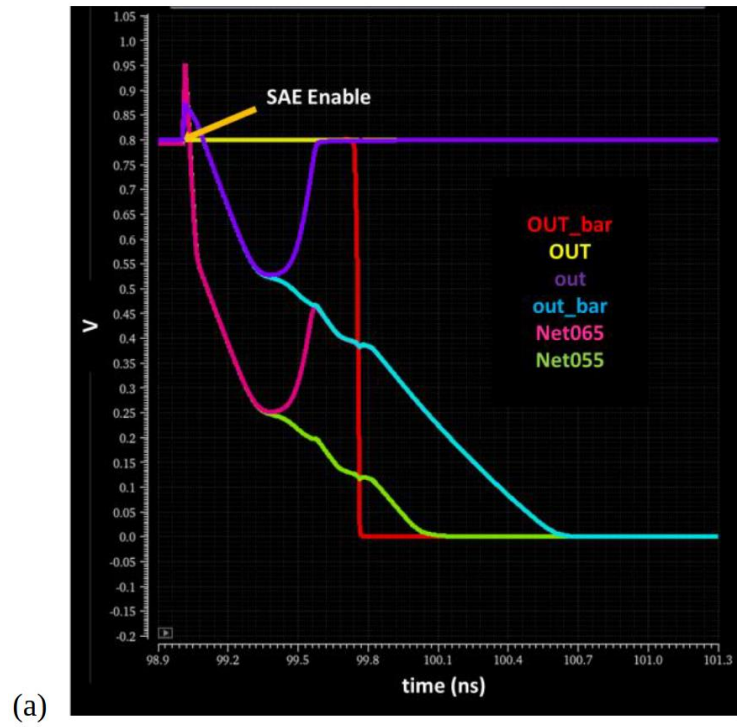


Fig. 20. ADC delay for (a) CSA and (b) VSA.

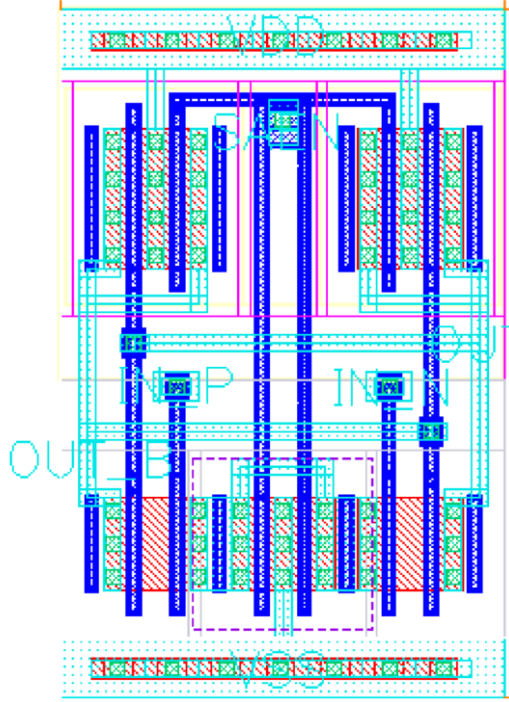


Fig. 21. CSA/VSA layout.

Area. The sense amplifier part of ADC area is calculated based on CSA/VSA layout in Fig. 21. For the sake of simplicity, we ensure the CSA/VSA design have the same layout, so that the same area equations can be used for the ADC area estimation. Reference resistances that are being compared with the column resistances are implemented using dummy columns. Dummy column has the same design as the main array (RRAM-based, SRAM-based ... etc.), but the supply voltages and the stored memory bits are varied flexibly to provide a desired equivalent resistance for the ADC operation. It is also assumed that dummy column can be shared amongst many different sense amplifiers to reduce the area overhead by the reference resistances. The parameter **dumcolshared** in **Param.cpp** determines how many sense amplifiers share one dummy column. The main advantage of such modeling approach is that the area estimation for dummy columns/arrays can be carried out simply using the same area equations as that of the main array.

Latency. The ADC latency is estimated as the worst-case CSA/VSA delay given the reference resistance sets, determined by the device on/off resistance and ADC bit precision. The worst-case CSA/VSA delay refers to the propagation delay between SAE and OUT/OUT_BAR signals (Fig. 20) when there is a 10% difference between the column resistance and the reference resistance. The final ADC delay for clock period (critical delay) estimation is set as the highest value among the worst-case CSA/VSA delays associated with each reference resistance. The ADC delay is the dominant component of the critical delay of the subarray, which are used to determine the clock frequency of the chip. Typical waveforms of several signals during ADC operations and worst-case ADC delay as a function of reference resistance is plotted in Fig. 20 (a)-(b). When SAE signal is turned on and V_{read} is applied, net 038/014 start to charge toward V_{read} . Consequently, M17 and M18 gradually become stronger and start to pull down the node voltages of net 055/065 towards ground. Depending on the charging speed difference between the net 038/014, the pulldown strengths of M17 and M18 and thus M7 and M6 differ. If the net 014 charging is faster than net 038, which is the case for the worst-case delay assumption, M18 becomes stronger than M17 and M7 becomes stronger than M6. As a result, some time after “SAE” enable “out_bar” starts to be pulled down to net 065 while “out” is being pulled up to V_{dd} in the latch (consisted of M6, M7, M11, and

M14). Thus, the final voltage of CSA/VSA output is essentially determined by the charging speed difference of net055/net065, affected by the resistance difference between reference and main array columns.

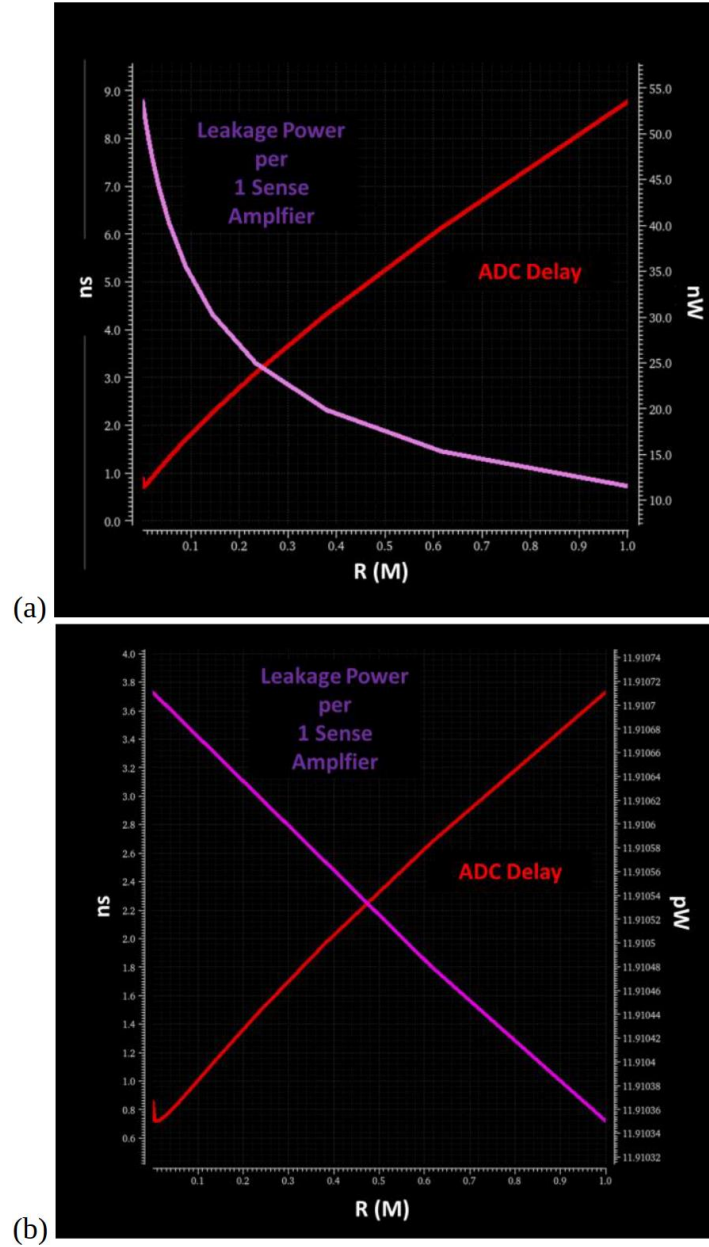


Fig. 22. (a) Worst-case CSA delay (red line) and static power/CSA (purple line) plotted against reference resistance. (b) Worst-case VSA delay (red line) and static power/VSA (purple line) plotted against reference resistance.

Power. ADC power can be decomposed into two categories: (i) dynamic energy consumption from the subarray column charging/discharging + transistor switching (ii) static energy consumption from the leakage power dissipation in reference/main array cells and transistors. (i) is estimated using the dynamic energy estimation formula using the node capacitance & voltage, consistent with the general modeling equations in other circuit modules. (ii) is estimated from the look up table containing static power/1 sense

amplifier as a function of resistance (can either be reference resistance or main array resistance), obtained from the Cadence simulation results. The generated worst-case CSA/VSA delay and static power consumption as a function of reference resistance values are plotted in Fig. 22 (a)-(b).

Analysis of the latency/power trend. It is worth noting some trends observed from Fig. 22 (a)-(b). It is expected that as the reference resistance values become higher, the ADC delay will also increase since the RC delay along the reference/main array column will increase. The increased RC delay slows down the development of the voltage difference between net 038/014 (and net 055/056), delaying the development of the final voltage in the latch of the sense amplifier. The expected tendency is well reflected in the plotted results in Fig. 22 (a)-(b). However, it is also observed that the ADC delay also increases when the reference resistance drops below a certain threshold. Such a counterintuitive trend observed from both CSA/VSA can be explained by the fact that the time required to develop a sufficient voltage difference between net038/014 increases when both reference/main array column charge faster beyond a certain threshold. Node voltages at net 038/014 reach close to V_{read} shortly after SAE enable when reference resistance/main array resistance values are sufficiently low, due to the low RC delay along column. Although the currents flowing through drain terminals of M17 & M18 are large, it takes longer time to develop final latch voltage as currents on reference/main array sides have small difference. Thus, the bottleneck of the ADC delay switches from the RC delay to charge the net 038/014 to the voltage difference development delay as the reference resistance constantly decreases towards 0.

According to Fig. 22 (a)-(b), the static power consumption from VSA is significantly less than that in CSA. This is because the `ref_voltage` in Fig. 19 (b), which sets the equivalent on-resistance M33 and M34 can be set to very low values in simulation without affecting the circuit functionality (For NeuroSim 1.4, `ref_voltage` is set to 0.05 V). The equivalent on resistance of M33 and M34 make the current flow through the drain terminals of M33 and M34 (which correspond to the reference/main array column current) negligibly small, resulting in several orders of magnitude less static power consumption in VSA. Due to the negligible power consumption in VSA, we set the static power consumption of VSA to 0 in NeuroSim 1.4.

To determine the ADC latency/power at different read voltages other than the default values, it is advised that users conduct SPICE simulation under desired read voltages based on the circuits in Fig. 19 (a)-(b) to modify the latency/power table in **MultilevelSenseAmp.cpp**.

6.2.8 Adder and register

As mentioned earlier, the adders and registers are used to accumulate the partial weighted sum results during the row-by-row weighted sum operation in digital synaptic array architectures. The group of adders is instantiated from **Adder** class in **Adder.cpp** and the group of registers (**dff**) is instantiated from **DFF** class in **DFF.cpp**.

6.2.9 Adder and shift register

The adder and shift register pair at the bottom of synaptic core performs shift and add of the weighted sum result at each input vector bit cycle (B_1 to B_n in Fig. 15 (b)) to get the final weighted sum. The bit-width of the adder and shift register needs to be further extended depending on the precision of input vector. If the values in the input vector are only 1 bit, then the adder and shift register pair is not required. In the simulator, a collection of the adder and shift register pairs (**ShiftAdd**) is instantiated from **ShiftAdd** class in **ShiftAdd.cpp**, where **ShiftAdd** further contains a group of adders (**adder**) instantiated from **Adder** class in **Adder.cpp** and a group of registers (**dff**) instantiated from **DFF** class in **DFF.cpp**.

7. Device Level: Technology File

Technology.cpp and Param.cpp contain the transistor dimension/performance metrics and standard cell design parameters for each technology node, required for estimating the PPA of the circuits. In NeuroSim 1.4, technology file (Technology.cpp) is updated to support down to 1 nm CMOS technology node. Stacked nanosheet devices and backside power rail are introduced in the code from 2 nm. The details of the device and standard cell design assumptions/parameters are summarized in Table II-III below:

		Technology Node (nm)						
		14	10	7	5	3	2	1
Metal track in M0 (T)		9	7.5	6	6	6	5.7	5
Metal pitch (nm)	M2	64	44	40	36	32	24	16
	M0	64	44	40	30	24	20	16
	M1	78	64	57	34	32	23	20
# of fins per standard cell (NMOS + PMOS)		4+4	3+3	2+2	2+2	2+2	1+1	1+1
Cell height (nm)		576	330	240	180	144	114	80
Contacted poly pitch (nm)		78	64	57	51	48	45	40
PN separation length (nm)		136	100	83	64	45	40	15
Fin pitch (nm)		48	36	30	28	24	26	24
Barrier thickness for M0-M2 (nm)		2.5	2.5	2.5	2.0	1.5	0.5	0.5

Table II. Assumed standard cell design assumptions for NeuroSim.

	Technology Node (nm)						
	14	10	7	5	3	2	1
Fin height [FH] (nm)	42	45	50	50	48	-	-
Fin width [FW] (nm)	8	8	7	6	5	-	-
NS thickness [NT] (nm)	-	-	-	-	-	6	6
NS width [NW] (nm)	-	-	-	-	-	15	10
NS vertical pitch [VP] (nm)	-	-	-	-	-	14 IRDS: 18	12 IRDS: 16
# of stacked NSs	-	-	-	-	-	3.0	4.0
Gate length (nm)	26	22	22	20	18	14	12
Equivalent oxide thickness [EOT] (nm)	0.9	0.8	0.7	0.65	0.6	0.55	0.5
Gate height over fin [GHOF] (nm)	40.0	35.0	30.0	25.0	20.0	10.0 IRDS: 15	5.0 IRDS: 10
Spacer width [SW] (nm)	12.0	8.0	8.0	7.0	6.0	6.0	5.0
Spacer k value	4.5	4.5	4.5	4	3.5	3.3	3
Contact resistivity ($\Omega \cdot \text{cm}^2$)	2e-9	2e-9	2e-9	2e-9	2e-9	5e-10	5e-10
On current/fin (μA)	54.744	58.725	60.139	61.320	64.788	66.385	59.005
On current density ($\mu\text{A}/\mu\text{m}$)	595.045	599.237	562.048	578.495	641.463	526.868	460.980
Off current/fin (pA)	9.856	12.516	15.752	14.676	16.006	9.242	21.747
g_m/fin (mS)	0.130	0.177	0.191	0.193	0.204	0.248	0.307
Supply voltage (V)	0.800	0.750	0.700	0.700	0.700	0.650	0.600
Gate capacitance (nF/m)	1.128	0.995	0.939	0.772	0.719	0.633	0.523
Junction capacitance (F/m^2)	0.012	0.013	0.014	0.012	0.013	0.009	0.010
Doping Concentration							
PTS doping (cm^{-3})	7e18						
Substrate doping (cm^{-3})	3e17						
S/D doping (cm^{-3})	2e20						
S/D extension doping (cm^{-3})	1e18						
Channel doping (cm^{-3})	1e15						

Table III. Assumed device parameters for NeuroSim.

Standard cell dimension scaling is the driving force of the Moore's law. We assume that FinFET and stacked nanosheet are adopted at 14 nm to 3 nm and at 2 nm/1 nm, respectively. Standard cell is largely characterized by two parameters: contacted poly pitch (CPP) and the cell height (CH). CH can be decomposed into three components: PN separation length, Active area, Outer area. We determine PN separation length by assuming two dummy fins are used for the separation of PMOS and NMOS for 14-7 nm and using International Roadmap for Devices and Systems (IRDS) 2021-2022 [13] projection values for 5-1 nm nodes. CPP, CH, and fin pitch (FP) for 14-7 nm are taken from the reported major company data [14-20] and the IRDS 2021-2022 values corresponding to the high-density (HD) application are used for 5-1 nm nodes. Fin number/standard cell of 14-7 nm nodes follow the reported industry trends [21-23] and that for 5-3 nm node is assumed to be 2+2/standard cell (NMOS+PMOS/standard cell). M0/M1/M2 metal pitch for 14-7 nm is assumed to be equal to minimum metal pitch/CPP/minimum metal pitch from

the reported major company data [14-20] and that for 5-1 nm nodes follow IRDS 2021-2022's projections. M0/M1/M2 metal width is assumed to be one-half of the M0/M1/M2 metal pitch. The standard cell parameters used for the extended NeuroSim are summarized in Table 1 (in the appendix).

We use Sentaurus TCAD simulation to generate the device parameters. Due to the limited accessibility to the actual foundry data for the advanced technology nodes (14-1 nm), we follow IRDS 2016-2022's projection data [13] to set the on-current density. We set target I_{on} ($\mu A/fin$) as the value corresponding to the HD application. Using the device design parameters as shown in Table III, we build an NMOS transistor at each technology node using Sentaurus TCAD simulation (NMOS and PMOS are assumed to have symmetrical characteristics). Device parameters such gate capacitance (normalized to effective width) and junction capacitance (normalized to S/D junction area), and transconductance g_m which are necessary for NeuroSim hardware estimation are extracted from the simulated I_D-V_G , $C_{GG}-V_G$ curve from Sentaurus TCAD.

It should be noted that updated technology parameters beyond 14 nm are based on the limited information provided from papers by major companies and IRDS projection. User can utilize their own judgement to modify the technology parameters in Technology.cpp if necessary.

8. Algorithm Level: PyTorch Wrapper

In NeuroSim V1.5, the framework utilizes NVIDIA's TensorRT for post-training quantization (PTQ) to prepare neural network models for Compute-in-Memory (CIM) simulation [22]. This replaces the WAGE quantization method [26] used in previous versions. The integration with TensorRT offers significant advantages, primarily the ability to quantize arbitrary, pre-trained models (including transformers) from standard libraries without requiring model retraining.

The process leverages TensorRT's capabilities for automatic operator replacement, converting standard floating-point layers (like linear and convolutional) into their quantized integer equivalents suitable for CIM mapping. Users can configure the desired bit-widths for network inputs (activations) and weights. TensorRT determines the optimal scaling factors needed for converting floating-point ranges to fixed-point integers using calibration techniques based on analyzing the distribution of activations and weights on a small calibration dataset. NeuroSim V1.5 supports TensorRT's **max calibration** (using the maximum observed absolute value) or **histogram calibration** (which analyzes value distributions to set scaling factors, potentially ignoring outliers) methods to find these optimal scaling factors.

The output of this stage is a quantized model where activations and weights are represented as integers. These quantized values and their precisions are crucial inputs for the NeuroSim framework. Specifically:

- **Weight Precision:** Determines the number of states required per synapse and influences the memory cell design and mapping strategy (e.g., number of cells per weight) in the hardware estimator.
- **Activation Precision:** Dictates the number of cycles required for processing inputs when using a bit-serial input scheme in the hardware simulation.

These parameters, derived from the TensorRT quantization process, bridge the algorithm level with the hardware simulation core, enabling the estimation of PPA metrics based on the specific precision requirements of the quantized network.

9. Algorithm Level: Inference Accuracy Estimation

While the hardware estimator calculates Power, Performance, and Area (PPA), the PyTorch wrapper estimates the inference accuracy of the quantized model mapped onto the CIM hardware, considering various non-ideal hardware effects. The neural network model (weights) is assumed to be pre-trained off-chip and then mapped to the CIM inference chip using the TensorRT quantization flow described in Section 8. Effects related to weight updates (nonlinearity, asymmetry, endurance) are not modeled in this inference simulation but are considered in training-focused versions of NeuroSim (V2.0-V2.2).

NeuroSim V1.5 introduces a flexible framework for modeling the impact of hardware non-idealities on inference accuracy, primarily through two distinct modes, visualized in Fig. 23: **Device Expert Mode** and **Circuit Expert Mode**. Users select the mode based on the type of noise data available and the desired level of modeling detail. The general procedure for behavioral simulation including noise injection is shown in Fig. 24.

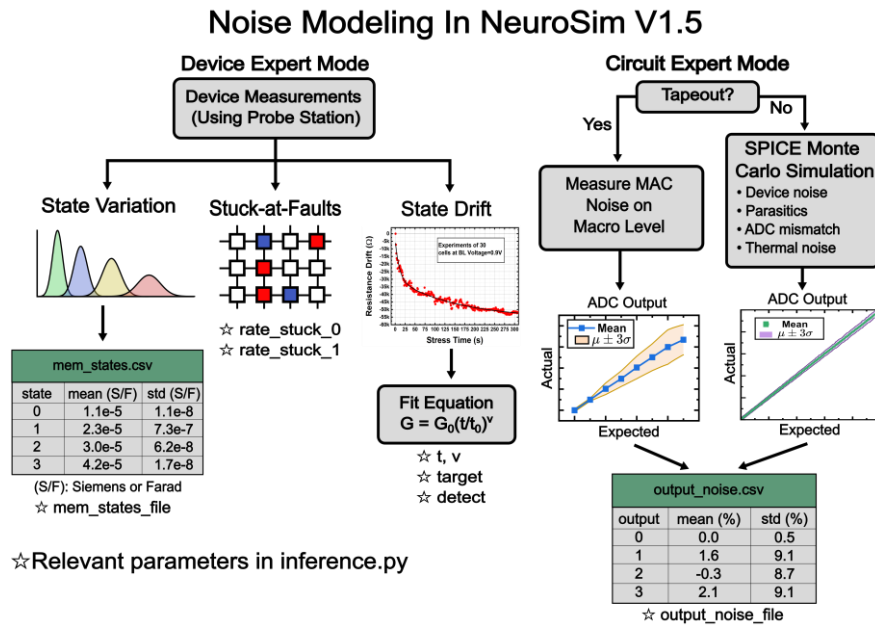
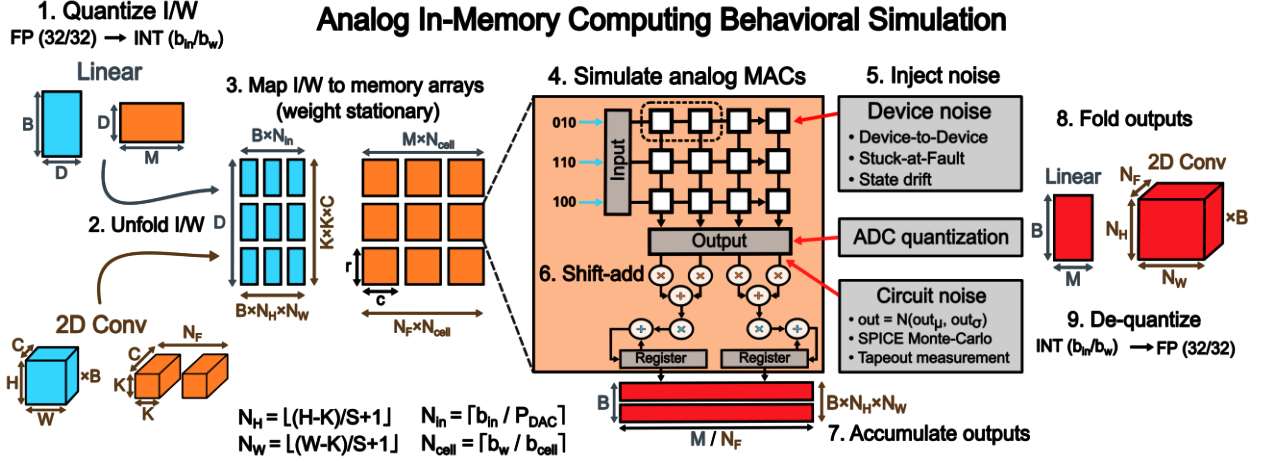


Fig. 23. NeuroSim V1.5 flowchart. Device expert mode: user provides device data (memory state variation, stuck-at-faults, state drift). Circuit expert mode: user provides statistical data on MAC outputs from SPICE Monte-Carlo simulations or tapeout measurements.



b_{in} : bits per input | b_w : bits per weight | b_{cell} : bits per cell (MLC) | N_{cell} : cells per weight | N_{in} : cycles per input | S: stride *PPA supports $P_{DAC}=1$ only

Fig. 24. Procedure for ACIM behavioral simulation in NeuroSim V1.5. (1) Inputs and weights are quantized from floating point to integer. (2) Inputs and weights unfolded and (3) mapped to compute-in-memory arrays. (4) Analog MACs are calculated with analytical circuit models. (5) Noise is injected either to memory devices or to digital MAC outputs. (6) ADC outputs are shifted and added. (7) Outputs from memory arrays are accumulated and concatenated. (8) Accumulated outputs are folded into output tensors. (9) Outputs are de-quantized from integer back to floating-point.

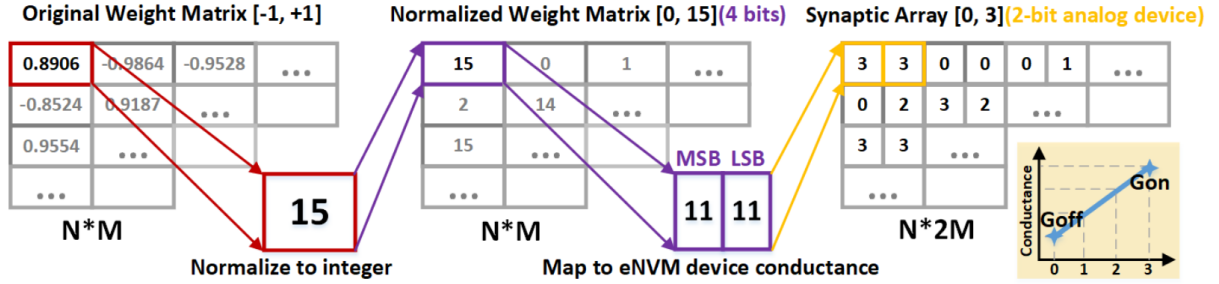


Fig. 25. Mapping weight from algorithm to synaptic device conductance in CIM architecture.

As Fig. 25 shows, to represent the weights from algorithm (floating-point) on the CIM architectures, due to the limited precision of synaptic devices, one ideal way is to normalize the weights to decimal integers, and then digitalize the integers to conductance levels. For example, as shown in Fig. 25, if we define the synaptic weight precision to be 4-bit (decimal integer 0 to 15), and represented by 2-bit (conductance level 0 to 3) synaptic devices, from algorithm, the floating-point weight “+0.8906” will be normalized to 15, and thus be mapped to two synaptic devices, one as LSB and one as MSB, and each of them are on conductance level 3 (i.e. $15/4=3$, $15\%4=3$).

9.1 Noise Modeling: Device Expert Mode

This mode allows users to model the impact of specific, individual device-level physical effects. It is activated by default when the Circuit Expert Mode (see Section 9.2) is not explicitly enabled (i.e., when `args.output_noise` is set to 0). Key non-idealities modeled in this mode include:

9.1.1 Conductance On/Off Ratio

Ideally, the conductance levels of synaptic devices range from 0 to 2^N , where N is the precision of synaptic devices. However, the minimum conductance can be regarded as 0 only if the conductance on/off ratio (=maximum conductance/minimum conductance) of synaptic devices is infinity, which is not feasible in current technology.

Thus, in reality, the minimum conductance level is not an ideal “0”. For example, if we use a normalized synaptic device conductance range as $0 \sim 1$ (as $0 \sim 2^N/2^N$), where the “1” can be represented as maximum conductance, and “0” is minimum conductance, in algorithm aspect, the conductance level “1” represent ideal “1”, while the conductance level “0” actually represent a non-zero value “ $1/(\text{on/off ratio})$ ”. In this case, small on/off ratio will introduce such non-ideal zeros into the calculation, and significantly distort the inference accuracy.

One approach to remedy this situation is to eliminate the effect of the OFF-state current in every weight element with the aid of a dummy column. In this framework, as Fig. 26 shows, we map the algorithm weights (range $[-1, +1]$) to synaptic devices (conductance range $[G_{\min}, G_{\max}]$) in the synaptic arrays, while we set a group of dummy columns beside each synaptic array, and the devices in dummy columns are set to the middle conductance $G_{\min}$. Such that, by subtracting the real outputs with the dummy outputs, the truncated conductance range will be $[0, G_{\max} - G_{\min}]$, and the off-state current effects are perfectly removed.

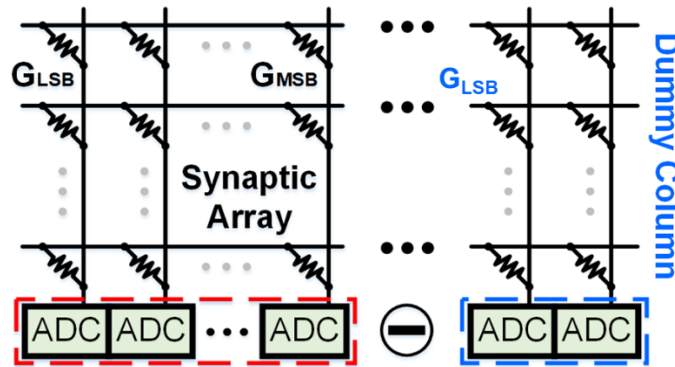


Fig. 26. Dummy column scheme to cancel off-state current effects.

In addition, users can specify the device conductance and standard deviation in a dedicated .csv file called “mem_states.csv” using “args.mem_states_file”. In this file, each device state is defined by 1) the logic state (‘0’ or ‘1’), 2) mean conductance, 3) and absolute standard deviation value. (Please check the example file “mem_states.csv” for details.) Note that if the csv file is specified in the input argument, the simulator does not count the on_state and off_state parameters. Please set args.mem_states_file to “” if the user prefers on_state and off_state.

Table IV mem_states.csv.

state	mean (S/F)	std (S/F)
0	1.1e-5	1.1e-8

1	2.3e-5	7.3e-8
2	3.0e-5	6.2e-8
3	4.2e-5	1.7e-8

(S/F): Siemens or Farad

9.1.2 Conductance Variation (Device-to-Device - D2D)

Real-world synaptic devices exhibit variations in their conductance/capacitance levels. This D2D variation is modeled assuming a Gaussian distribution for each programmable state. Users provide the mean and absolute standard deviation for each state (e.g., '0' and '1' for binary devices, or multiple levels for MLC) via a CSV file specified by `args.mem_states_file` (see Table IV for format). If `args.mem_states_file` is not provided, the simulator may use default `on_state` and `off_state` parameters (check implementation). Each device's conductance/capacitance is sampled once from its state's distribution during initialization to emulate D2D variation.

9.1.3 Read Noise (Cycle-to-Cycle Variation)

Temporal fluctuations during readout can also occur. This is modeled as relative Gaussian noise added to the device's conductance/capacitance value during each compute operation. Users can specify the relative standard deviation using the `args.read_noise` parameter. Note that D2D variation (from `mem_states.csv`) and read noise (`args.read_noise`) can be enabled simultaneously.

9.1.4 Retention

Retention refers to the ability of memory device to retain its programmed state over a long period of time. Typical retention specification for NVM in memory application is more than 10 years at 85°C. Many binary eNVM devices have been able to meet this requirement. However, there are no reported data for analog eNVM that shows such retention, which can be attributed to the instability of intermediate conductance states.

To be general, we consider four scenarios of conductance drift for the retention analysis, as show in Fig. 27, where the conductance can either drift toward its maximum, minimum or intermediate states, or just randomly drift. The formula for modeling the conductance drift behavior is assumed to follow the one shown below:

$$G=G_0 \left(\frac{t}{t_0} \right)^v$$

where “ G_0 ” is the initial conductance, “ t ” is the retention time, “ v ” is the drift coefficient and “ t_0 ” is the time constant which is assumed to be 1 second in this framework.

To estimate the retention effect on inference accuracy, we define a function called “`calc_drift`” in file “`macro.py`”, where the retention time and drift coefficient are defined as “`args.t`” and “`args.v`” separately, while “`args.detect`” is used to define the drift scenario, if “`args.detect`” is 1, then the drift scenario is drifting to a fixed value (otherwise, it is random drift), and the targeted value is then defined as “`args.target`”, the range is defined from 0 to 1. Note that currently, each state is assumed to drift at the same speed defined by “`args.v`”. Those arguments can be defined in file “`inference.py`”.

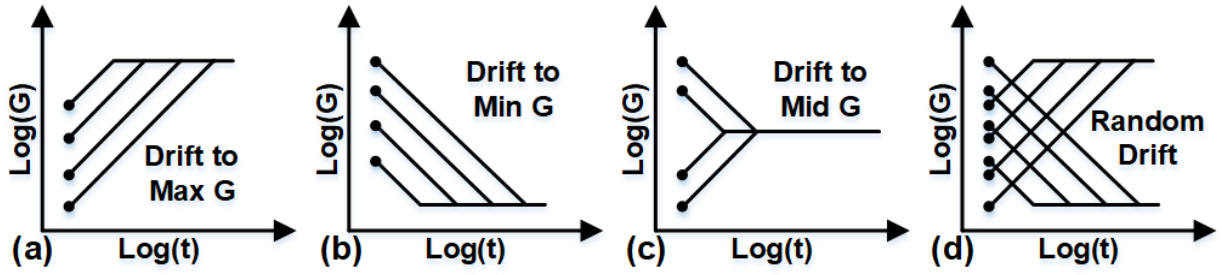


Fig. 27. Different scenarios of conductance drift.

9.1.5 Stuck-at-fault

Stuck-at-Fault (SAF) is one of the device non-idealities in non-volatile memory, where the device may become permanently fixed to their minimum or maximum states due to manufacturing defects or programming failures. This hard fault is modeled by the parameters “args.rate_stuck_0” and “args.rate_stuck_1” ranging from 0-1, representing the SAF rate for off state and on state, respectively. During initialization, each memory cell is set to either normal state or permanent fault state based on the SAF rate.

9.2 Noise Modeling: Circuit Expert Mode

This mode is designed for users who have pre-characterized statistical data representing the *aggregated* non-idealities of the entire analog MAC operation (including device variations, parasitics, thermal noise, circuit variations, etc.), typically obtained from extensive SPICE Monte Carlo simulations or direct silicon measurements of a fabricated CIM macro. This mode models noise at the *digital output* of the ADC.

Users activate this mode by setting the args.output_noise parameter:

- **Using output_noise.csv (args.output_noise = -1):** Provide a CSV file named output_noise.csv (see Table V for format). This file contains the mean and standard deviation of the *digital* ADC output for each possible *ideal* digital MAC result. The simulator directly samples from these distributions to determine the final output, effectively capturing the measured or simulated circuit behavior.
- **Using Uniform Noise (args.output_noise > 0):** Specify a single absolute standard deviation value via args.output_noise. This standard deviation is applied as Gaussian noise to all ADC output levels.

Important: When Circuit Expert Mode is enabled (args.output_noise > 0 or args.output_noise = -1), the individual device-level noise factors described in Device Expert Mode (D2D Variation, Read Noise, SAF, Retention) are **disabled** and bypassed in the simulation. This mode allows for rapid evaluation using high-level statistical circuit data.

Table V output_noise.csv

output	mean (%)	std (%)
0	0.0	0.5
1	1.6	9.1

2	-0.3	8.7
3	2.1	9.1

Users can refer to “inference.py” and “pytorch-quantization/pytorch_quantization/cim/modules/macro.py” for more implementation details.

9.3 ADC Quantization Effects

For CIM architecture, there are mainly two read-out schemes. A sequential processing method of the matrix-vector multiplication is to read out the dot-products in a row-by-row manner, which leads to extra energy and latency for accumulations along the rows. A more efficient method is parallel processing, where multiple rows are activated simultaneously by a switch matrix, and the current summation will be read out by an ADC. Therefore, the row-by-row accumulation periphery of sequential scheme is eliminated. However, since it is impractical to use very high-precision ADC at the edge of eNVM sub-arrays, we have to truncate the precision of ADC (for partial sums) to minimize the area and energy overhead.

To minimize the ADC precision while guarantee the inference accuracy, it is necessary to run the simulation of ADC quantization before hardware design. For V1.5, currently we only support linear quantization as Fig. 28 shows, the ADC references are distributed linearly across possible partial-sum value range in the synaptic array. For different ADC precisions which can be set by “args.adc_precision”, the quantization interval is fixed. Decreasing the precision decreases the upper bound and simply clips any output larger than the upper bound. We found this scheme to strike a good balance between simplicity and accuracy. From our tests, this method achieved the same accuracy as keeping the bounds fixed while changing the quantization interval. We encourage users to try other schemes.

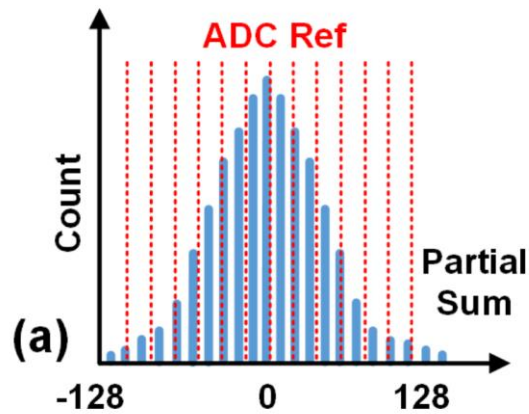


Fig. 28. ADC quantization.

(Note: The parameter “args.dac_precision” only supports accuracy and software runtime testing. DAC is not modeled in the hardware PPA evaluation, and the hardware only evaluates bit-serial input for CIM operations.)

10. How to run *DNN +NeuroSim*

10.1 Define the network and dataset

Firstly, the users have to define network structure and load the trained neural network model in `quantize.py`. Pretrained models from Pytorch Official or Hugging Face are supported for simulation. After that, configure the dataset loading in `dataset.py`. A network topology table (`NetWork.csv`) will be generated and saved in `/NeuroSIM` folder automatically. Table VI shows an example of the VGG8 topology table.

Table VI NetWork.csv

	IFM Length	IFM Width	IFM Channel Depth	Kernel Length	Kernel Width	Kernel Depth	Followed by pooling or not?
Layer 1	32	32	3	3	3	128	0
Layer 2	32	32	128	3	3	128	1
Layer 3	16	16	128	3	3	256	0
Layer 4	16	16	256	3	3	256	1
Layer 5	8	8	256	3	3	512	0
Layer 6	8	8	512	3	3	512	1
Layer 7	1	1	8192	1	1	1024	0
Layer 8	1	1	1024	1	1	10	0

In the default VGG-8 network, layer 1 to layer 6 are convolutional layers, and layer 7 to layer 8 are fully-connected layers. In the Table IV, the dimensions of each layer are defined in different rows, from layer 1 to layer 8 (row 1 to row 8), while the first three columns (column 1 to column 3) are used to define the dimension of input feature maps (IFMs) of each layer. For example, the input image size of layer 1 is $32 \times 32 \times 3$, thus, in first row, the first three cells should be filled by 32, 32 and 3 respectively, which indicated the length, width and depth of the IFM. The next three columns (column 4 to column 6) are used to define the dimension of kernels. For example, the kernel size of layer 3 is $3 \times 3 \times 128 \times 256$ (i.e. each single 3D kernel is $3 \times 3 \times 128$, the kernel depth is 256), since it is well known that the third dimension of kernel is defined by the IFM channel depth, it is not necessary to define the third dimension again, thus, from the Table IV, in row 3, the fourth, fifth and sixth cell should be filled by 3, 3 and 256, which represent the length, width and kernel depth (first, second and fourth dimension of kernel) respectively. One should notice that the fully-connected layer can also be represented in the similar way, by considering it as a special convolutional layer, which has unit length and width for IFM and kernels. The last column is used to define whether the current layer is followed by pooling, it will be read by *NeuroSim*, and properly estimate the hardware performance for pooling function, in this framework, the activation function is considered to be integrated in every layer.

10.2 Modify the hardware parameters in `Param.cpp`

After setting up the network structure, the users need to define the hardware parameters in **`Param.cpp`**. In this file, the users could define the parameters, such as technology node (**`technode`**), device type (**`memcelltype`**: SRAM, RRAM, FeFET, or nvCap), operation mode (**`operationmode`**: parallel or sequential analog, synaptic sub-array size (**`numRowSubArray`**, **`numColSubArray`**), synaptic device precision (**`cellBit`**), mapping method (conventional or novel), activation type (sigmoid or ReLU), cell height/width in feature size (F), clock frequency and so on. We list some recommended device parameters as below:

Table VI Scaling trend of SRAM cell area with technology nodes (assuming F is the same as the technology node)

SRAM	22nm	14nm	10nm	7nm	5nm	3nm	2nm	1nm
8T SRAM cell size (F^2)	360	480	720	1080	-	-	-	-
6T SRAM cell size (F^2)	280	326	420	551	1120	2048	4680	11500

Table VIII Recommended eNVM device parameters.

	22nm RRAM (Intel) [26]	90nm RRAM (Winbond) [37]	130nm RRAM (Tsinghua) [38]	90nm PCM (IBM) [49]	22nm FeFET (GF) [50]	22nm STT- MRAM (Intel) [51]
Ron (Ω)	6k	6k	100k	40k	240k	1.4k
On/off ratio	17	150	10	12.5	100	2.8
Cell size (F^2)	4×12	6×6	4×4	4×4	4×6	10×10

In this framework, all the hardware parameters that users need to define are summarized in the **Param.cpp**, thus, to successfully run the simulator, the two main files users need to visit are **NetWork.csv** and **Param.cpp**.

10.3 Compilation of NeuroSim

After modifying the **Param.cpp** files, or whenever any change is made in the files, the codes have to be recompiled by using `make` command as stated in Installation and Usage (Linux + Anaconda) section. If the compilation is successful, a screenshot like Fig. 25 can be expected.

10.4 Run the program with PyTorch wrapper

```
g++ -c -fopenmp -O3 -std=c++0x -W NewMux.cpp -o NewMux.o
g++ -c -fopenmp -O3 -std=c++0x -W ProcessingUnit.cpp -o ProcessingUnit.o
g++ -c -fopenmp -O3 -std=c++0x -W Bus.cpp -o Bus.o
g++ -c -fopenmp -O3 -std=c++0x -W XorArbiterPuf.cpp -o XorArbiterPuf.o
g++ -c -fopenmp -O3 -std=c++0x -W ArbiterPuf.cpp -o ArbiterPuf.o
g++ -c -fopenmp -O3 -std=c++0x -W formula.cpp -o formula.o
g++ -c -fopenmp -O3 -std=c++0x -W DFF.cpp -o DFF.o
g++ -c -fopenmp -O3 -std=c++0x -W FunctionUnit.cpp -o FunctionUnit.o
g++ -c -fopenmp -O3 -std=c++0x -W RippleCounter.cpp -o RippleCounter.o
g++ -c -fopenmp -O3 -std=c++0x -W Adder.cpp -o Adder.o
g++ -c -fopenmp -O3 -std=c++0x -W Technology.cpp -o Technology.o
g++ -c -fopenmp -O3 -std=c++0x -W MultilevelSenseAmp.cpp -o MultilevelSenseAmp.o
g++ -c -fopenmp -O3 -std=c++0x -W SwitchMatrix.cpp -o SwitchMatrix.o
g++ -c -fopenmp -O3 -std=c++0x -W CurrentSenseAmp.cpp -o CurrentSenseAmp.o
g++ -c -fopenmp -O3 -std=c++0x -W NewSwitchMatrix.cpp -o NewSwitchMatrix.o
g++ -c -fopenmp -O3 -std=c++0x -W WLNwDecoderDriver.cpp -o WLNwDecoderDriver.o
g++ -c -fopenmp -O3 -std=c++0x -W BitShifter.cpp -o BitShifter.o
g++ -c -fopenmp -O3 -std=c++0x -W VoltageSenseAmp.cpp -o VoltageSenseAmp.o
g++ -c -fopenmp -O3 -std=c++0x -W MaxPooling.cpp -o MaxPooling.o
g++ -c -fopenmp -O3 -std=c++0x -W Sigmoid.cpp -o Sigmoid.o
g++ -c -fopenmp -O3 -std=c++0x -W Param.cpp -o Param.o
g++ -c -fopenmp -O3 -std=c++0x -W SubArray.cpp -o SubArray.o
g++ -c -fopenmp -O3 -std=c++0x -W AdderTree.cpp -o AdderTree.o
g++ -c -fopenmp -O3 -std=c++0x -W Comparator.cpp -o Comparator.o
g++ -c -fopenmp -O3 -std=c++0x -W DecoderDriver.cpp -o DecoderDriver.o
g++ -c -fopenmp -O3 -std=c++0x -W Subtractor.cpp -o Subtractor.o
g++ -c -fopenmp -O3 -std=c++0x -W MultilevelSAEncoder.cpp -o MultilevelSAEncoder.o
g++ -c -fopenmp -O3 -std=c++0x -W Chip.cpp -o Chip.o
g++ -c -fopenmp -O3 -std=c++0x -W Precharger.cpp -o Precharger.o
g++ -c -fopenmp -O3 -std=c++0x -W RowDecoder.cpp -o RowDecoder.o
g++ -c -fopenmp -O3 -std=c++0x -W Mux.cpp -o Mux.o
g++ -c -fopenmp -O3 -std=c++0x -W SenseAmp.cpp -o SenseAmp.o
g++ -c -fopenmp -O3 -std=c++0x -W Buffer.cpp -o Buffer.o
g++ -c -fopenmp -O3 -std=c++0x -W WLDcOutput.cpp -o WLDcOutput.o
g++ -c -fopenmp -O3 -std=c++0x -W ReadCircuit.cpp -o ReadCircuit.o
g++ -c -fopenmp -O3 -std=c++0x -W DeMux.cpp -o DeMux.o
g++ -c -fopenmp -O3 -std=c++0x -W Tile.cpp -o Tile.o
g++ -c -fopenmp -O3 -std=c++0x -W ShiftAdd.cpp -o ShiftAdd.o
g++ -c -fopenmp -O3 -std=c++0x -W HTree.cpp -o HTree.o
g++ -c -fopenmp -O3 -std=c++0x -W SRAMWriteDriver.cpp -o SRAMWriteDriver.o
g++ -c -fopenmp -O3 -std=c++0x -W SramNewSA.cpp -o SramNewSA.o
g++ -c -fopenmp -O3 -std=c++0x -W main.cpp -o main.o
g++ -fopenmp -O3 -std=c++0x -W NewMux.o ProcessingUnit.o Bus.o XorArbiterPuf.o ArbiterPuf.o formula.o DFF.o FunctionUnit.o RippleCounter.o Adder.o Technology.o MultilevelSenseAmp.o SwitchMatrix.o CurrentSenseAmp.o NewSwitchMatrix.o WLNwDecoderDriver.o BitShifter.o VoltageSenseAmp.o MaxPooling.o Sigmoid.o Param.o SubArray.o AdderTree.o Comparator.o DecoderDriver.o Subtractor.o MultilevelSAEncoder.o Chip.o Precharger.o RowDecoder.o Mux.o SenseAmp.o Buffer.o WLDcOutput.o ReadCircuit.o DeMux.o Tile.o ShiftAdd.o HTree.o SRAMWriteDriver.o SramNewSA.o main.o -o main
```

Fig. 29 Output of compilation.

After compilation of *NeuroSim*, go back to the PyTorch wrapper. In the wrapper, there are four networks (VGG-8 network for CIFAR-10 dataset, ResNet-18 network for CIFAR-100 dataset, ResNet-50 network for ImageNet dataset, and swin transformer for ImageNet dataset) as default, the users can modify their network structures, and run the simulator correspondingly. Use command *python inference.py* to run the simulation.

The program will print out the results for each layer of the network during the simulation. The simulation will approximately take 6 minutes with a computer workstation (Intel 8-core CPU with 3.2 GHz and NVidia GPU A6000) for the VGG-8 network. Fig. 30 shows an example of final output of an 8-bit VGG-8 inference for one CIFAR-10 image, based on parallel non-volatile capacitor array, with 1-bit per cell. The outputs from the simulation include hardware inference accuracy, memory utilization, and latency/energy/leakage breakdown for 1-image inference, and the equivalent energy efficiency in terms of TOPS/W, and throughput in terms of TOPS and frames per second (FPS), and compute efficiency in terms of TOPS/mm².

```

76% | 19/25 [03:48:01:11, 11.95s/it]
Test Accuracy of the model on 8000 test Images: 89.45 %
Average Test loss: 10.28257367014885

80% | 20/25 [04:00:00:59, 11.95s/it]
Test Accuracy of the model on 8400 test Images: 89.46428571428571 %
Average Test loss: 10.69843184421616

84% | 21/25 [04:12:00:47, 11.95s/it]
Test Accuracy of the model on 8800 test Images: 89.3409090909091 %
Average Test loss: 11.249496787786484

88% | 22/25 [04:24:00:35, 11.95s/it]
Test Accuracy of the model on 9200 test Images: 89.43478268869566 %
Average Test loss: 11.627468436956486

92% | 23/25 [04:36:00:23, 11.95s/it]
Test Accuracy of the model on 9600 test Images: 89.47916666666667 %
Average Test loss: 12.057075589895248

96% | 24/25 [04:48:00:11, 11.95s/it]
Test Accuracy of the model on 10000 test Images: 89.45 %
Average Test loss: 12.560415416955948

-----
evaluation time: 299.9981427192688
96% | 24/25 [04:59:00:12, 12.50s/it]
--- Hardware Properties ---
subarray size:
[128, 128]
parallel read:
128
ADC precision:
7
cell precision:
1
on/off ratio:
10.0
User assigned speed-up degree is larger than the upper bound (where no idle period during the whole process)
The speed-up degree is auto-assigned as the upper bound (where no idle period during the whole process)
----- FloorPlan -----

The speed-up degree is auto-assigned as the upper bound (where no idle period during the whole process)
----- FloorPlan -----
Tile and PE size are optimized to maximize memory utilization ( = memory mapped by synapse / total memory on chip)
Desired Conventional Happed Tile Storage Size: 3824x3824
Desired Conventional PE Storage Size: 512x512
Desired level Happed Tile Storage Size: 9632x512
User-defined Subarray Size: 128x128
----- # of tile used for each layer -----
Layer1: 2
Layer2: 8
Layer3: 4
Layer4: 8
Layer5: 8
Layer6: 8
Layer7: 84
Layer8: 1
----- Speed-up of each layer -----
Layer1: 16
Layer2: 16
Layer3: 4
Layer4: 4
Layer5: 2
Layer6: 1
Layer7: 1
Layer8: 8
----- Utilization of each layer -----
Layer1: 0.218938
Layer2: 1
Layer3: 1
Layer4: 1
Layer5: 1
Layer6: 1
Layer7: 1
Layer8: 0.625
Memory Utilization of Whole Chip: 98.1038 %
----- FloorPlan Done -----

***** Breakdown of Latency and Dynamic Energy *****
----- Summary -----
ChipArea : 9.38654e+00um^2
Chip total CM array : 99321um^2
Total IC Area on chip (Global and Tile/PE local): 4.37374e+00um^2
Total ADC (on S/A and precharger for SRAM) Area on chip : 2.05145e+00um^2
Total Accumulation Circuits (subarray level: adders, shiftAddrs; PE/Tile/Global level: accumulation units) on chip : 1.58083e+00um^2
Other Peripherals (e.g. decoders, mux, switchmatrix, buffers, pooling and activation units) : 4.51552e+00um^2
Chip clock period is: 29.7213ns
Chip pipeline-system-clock-cycle (per Image) is: 2.32538e+00ns
Chip pipeline-system readDynamicEnergy (per Image) is: 3.51263e+07pJ
Chip pipeline-system leakage Energy (per Image) is: 7.82896e+06pJ
Chip pipeline-system leakage Power (per Image) is: 3396.72uW
Chip pipeline-system buffer readDynamicEnergy (per Image) is: 1.83747e+06ns
Chip pipeline-system buffer readDynamicEnergy (per Image) is: 187813pJ
Chip pipeline-system IC readlatency (per Image) is: 373256ns
Chip pipeline-system IC readDynamicEnergy (per Image) is: 1.16006e+07pJ
***** Breakdown of Latency and Dynamic Energy *****
----- ADC (on S/A and precharger for SRAM) readlatency is : 186907ns
----- Accumulation Circuits (subarray level: adders, shiftAddrs; PE/Tile/Global level: accumulation units) readlatency is : 31393.1ns
----- Other Peripherals (e.g. decoders, mux, switchmatrix, buffers, IC, pooling and activation units) readlatency is : 2.2174e+00ns
----- ADC (on S/A and precharger for SRAM) readDynamicEnergy is : 1.68688e+07pJ
----- Accumulation Circuits (subarray level: adders, shiftAddrs; PE/Tile/Global level: accumulation units) readDynamicEnergy is : 3.81955e+06pJ
----- Other Peripherals (e.g. decoders, mux, switchmatrix, buffers, IC, pooling and activation units) readDynamicEnergy is : 1.44289e+07pJ
***** Breakdown of Latency and Dynamic Energy *****
----- Performance -----
Energy Efficiency TOPS/W (Pipelined Process): 23.5099
Throughput TOPS (Pipelined Process): 0.529734
Throughput FPS (Pipelined Process): 436.407
Compute efficiency TOPS/mm^2 (Pipelined Process): 0.000569574
----- Hardware Performance Done -----
----- Simulation Performance -----
Total Run-time of NeuroSim: 70 seconds

```

Fig. 30 Example of final output.

11. Hardware Estimation Examples

We provide hardware estimation results for a few cases (including the added technology nodes) under VGG8-CIFAR10 workload.

Here, assumed subarray size and multiply-and-accumulate (MAC) operation is 128×128 and 8b (weight) $\times$ 8b (activation), respectively. For the RRAM cases, we use an on/off ratio of 17 and for SRAM mode we use an ideal on/off ratio of infinity. The nvCap memory cell is set with an on/off ratio of 10. We encourage the users to carefully select parameters such as the on/off ratio in the SRAM mode based on their circuit design as we are still working on the simulations for this mode (see the note below). For swin transformer model, novel mapping is not applied due to the network structure. The rest of the parameters follow the default setting in the released code.

TABLE IX Hardware Estimation Examples.

Memory	Tech	On/Off	ADC	Array Size	Parallel	Area	TOPS	TOPS/W	TOPS/mm ²
--------	------	--------	-----	------------	----------	------	------	--------	----------------------

Type	Node	Ratio	Precision		Read	(mm ²)			
VGG-8 on CIFAR-10									
RRAM	22	17	7	128×128	128	120	6.56	20.6	0.05
nvCap	22	25	6	64×64	64	104	4.75	23.26	0.05
SRAM	5	∞*	7	128×128	128	20.3	8.07	47.1	0.4
ResNet-18 on CIFAR-100									
RRAM	22	17	7	128×128	128	393	15.1	19.7	0.04
nvCap	22	25	6	64×64	64	267	5.4	30.4	0.02
SRAM	5	∞*	7	128×128	128	65.9	18.6	47.3	0.28
ResNet-50 on ImageNet									
RRAM	22	17	7	128×128	128	323	7.5	19.5	0.02
nvCap	22	25	6	64×64	64	228	2.54	25.2	0.01
SRAM	5	∞*	7	128×128	128	55.4	9.21	46.2	0.166
Swin transformer on ImageNet									
RRAM	22	17	7	128×128	128	268	4.46	7.62	0.016
nvCap	22	25	5	32×32	32	240	3.05	12.25	0.013
SRAM	5	∞*	7	128×128	128	46.4	5.48	24.2	0.12

*Note 1: The table was generated on **05/01/25**. The table will be constantly updated as we make minor updates in the code.

*Note 2: Regarding 6T SRAM mode, it is important to acknowledge that our current approach relies on estimates for hardware parameters and inference accuracy. At this stage, we have not fully incorporated more intricate simulations to account for the unique non-idealities found in analog SRAM mode. We understand the significance of accurate simulations in research, and thus, we encourage users to consider NeuroSim as a building block that can be tailored to suit their research needs. As the NeuroSim platform evolves, we aim to continually enhance its capabilities, including more detailed simulations and refined estimations. Your feedback and contributions in this process are highly valued as we strive for continuous improvement and progress in the field. Should you have any queries or suggestions, please do not hesitate to reach out to our support team.

Thank you for using NeuroSim.

12. Reference

- [1]. P.-Y. Chen, X. Peng, S. Yu, "NeuroSim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2018.
- [2]. github.com/neurosim/MLP_NeuroSim_V3.0
- [3]. W. Li, S. Huang, X. Sun, H. Jiang, S. Yu, "Secure-RRAM: A 40nm 16kb compute-in-memory 354 macro with reconfigurability, sparsity control, and embedded security," *IEEE Custom Integrated Circuits Conference (CICC)*, 2021.

- [4]. Predictive Technology Model (PTM). Available at <http://ptm.asu.edu/>
- [5]. N. E. Weste and D. Harris, "CMOS VLSI Design – A Circuit and Systems Perspective, 4th edition," 2007.
- [6]. X. Peng, R. Liu and S. Yu, "Optimizing weight mapping and data flow for convolutional neural networks on RRAM based processing-in-memory architecture," IEEE International Symposium on Circuits and Systems (ISCAS), 2019.
- [7]. P.-Y. Chen, et al., "Technology-design co-optimization of resistive cross-point array for accelerating learning algorithms on chip," ACM/IEEE Design, Automation & Test in Europe Conference & Exhibition (DATE), 2015.
- [8]. W. Khwa et al., "A 65nm 4Kb algorithm-dependent computing-in-memory SRAM unit-macro with 2.3ns and 55.8TOPS/W fully parallel product-sum operation for binary DNN edge processors," IEEE International Solid State Circuits Conference (ISSCC), 2018.
- [9]. M. Jerry, et al., "Ferroelectric FET analog synapse for acceleration of deep neural network training," IEEE International Electron Devices Meeting (IEDM), 2017.
- [10]. X. Sun, S. Yin, X. Peng, R. Liu, J.-S. Seo, S. Yu, "XNOR-RRAM: A scalable and parallel resistive synaptic architecture for binary neural networks," ACM/IEEE Design, Automation & Test in Europe Conference (DATE), 2018.
- [11]. Y-C. Luo, J. Read, et al "A Cross-layer Framework for Design Space and Variation Analysis of Non-Volatile Ferroelectric Capacitor-Based Compute-in-Memory Accelerators," IEEE Asia South Pacific Design Automation Conference (ASP-DAC), 2023.
- [12]. Y-C. Luo, J. Hur, et al "Experimental Demonstration of Non-volatile Capacitive Crossbar Array for In-memory Computing," IEEE International Electron Devices Meeting (IEDM) 2021
- [13]. IRDS 2016-2022 reports [Online] Available: <https://irds.ieee.org/editions>
- [14]. "Samsung 14 nm FinFET." <https://www.samsung.com/us/sas/Business/Foundry14nm> (accessed May. 18, 2023).
- [15]. 14 nm lithography process [Online] Available: https://en.wikichip.org/wiki/14_nm_lithography_process
- [16]. 10 nm lithography process [Online] Available: https://en.wikichip.org/wiki/10_nm_lithography_process
- [17]. 7 nm lithography process [Online] Available: https://en.wikichip.org/wiki/7_nm_lithography_process
- [18]. D. Schor, "TSMC 7nm HD and HP Cells, 2nd Gen 7nm, And The Snapdragon 855 DTCO." [Online]. Available: <https://fuse.wikichip.org/news/2408/tsmc-7nm-hd-and-hp-cells-2nd-gen-7nm-and-the-snapdragon-855-dtco/>.
- [19]. D. Schor, "VLSI 2018: Samsung's 8nm 8LPP, a 10nm extension." [Online]. Available: <https://fuse.wikichip.org/news/1443/vlsi-2018-samsungs-8nm-8lpp-a-10nm-extension/>.
- [20]. M. Sharma, "TSMC 5 nm vs. Intel 10nm vs. GloFo 7 nm." [Online]. Available: <https://www.apothetech.com/tsmc-5-nm-vs-intel-10nm-vs-glofo-7-nm/>.
- [21]. K. Gibb, "Samsung's 14 nm LPE FinFET transistors." [Online]. Available: <https://www.eenewseurope.com/en/samsungs-14-nm-lpe-finfet-transistors/>.
- [22]. TensorRT pytorch-quantization documentation [Online]. Available: <https://github.com/NVIDIA/TensorRT/tree/release/10.9/tools/pytorch-quantization>
- [23]. I. Huang, "Huawei, Apple use TSMC but their 7nm SoCs are different." [Online]. Available: <https://www.eetasia.com/huawei-apple-use-tsmc-but-their-7nm-socs-are-different/>.
- [24]. Comparing SMIC 7nm vs TSMC 7nm [Online] Available: <https://files.static-nzz.ch/2022/08/30/ef873456-9644-4a36-b160-ec0d41853e68.pdf>
- [25]. S. Wu, et al. "Training and inference with integers in deep neural networks," arXiv: 1802.04680, 2018.

- [26]. github.com/boluoweifenda/WAGE
- [27]. github.com/stevenygd/WAGE.pytorch
- [28]. github.com/aaron-xichen/pytorch-playground
- [29]. P. Jain, U. Arslan, M. Sekhar, B.C. Lin, L. Wei, T. Sahu, J. Alzate-vinasco, A. Vangapaty, M. Meterelliyo, N. Strutt, A.B. Chen, "A 3.6 Mb 10.1 Mb/mm² Embedded Non-Volatile ReRAM Macro in 22nm FinFET Technology with Adaptive Forming/Set/Reset Schemes Yielding Down to 0.5 V with Sensing Time of 5ns at 0.7 V," IEEE International Solid-State Circuits Conference (ISSCC), 2019.
- [30]. W. He, S. Yin, Y. Kim, X. Sun, J.J. Kim, S. Yu and J.S. Seo, "2-Bit-per-Cell RRAM based In-Memory Computing for Area-/Energy-Efficient Deep Learning," IEEE Solid-State Circuits Letters, vol. 3, pp. 194-197, 2020.
- [31]. W. Wu, H. Wu, B. Gao, P. Yao, X. Zhang, X. Peng, S. Yu, H. Qian, "A methodology to improve linearity of analog RRAM for neuromorphic computing," IEEE Symposium on VLSI Technology (VLSI), 2018.
- [32]. W. Kim, R.L. Bruce, T. Masuda, G.W. Fraczak, N. Gong, P. Adusumilli, S. Ambrogio, H. Tsai, J. Bruley, J.P. Han, M. Longstreet, "Confined PCM-based analog synaptic devices offering low resistance-drift and 1000 programmable states for deep learning," IEEE Symposium on VLSI Technology, 2019.
- [33]. K. Ni, B. Grisafe, W. Chakraborty, A.K. Saha, S. Dutta, M. Jerry, J.A. Smith, S. Gupta, S. Datta, "In-memory computing primitive for sensor data fusion in 28 nm HKMG FeFET technology," IEEE International Electron Devices Meeting (IEDM), 2018.
- [34]. L. Wei, J.G. Alzate, U. Arslan, et al. "A 7Mb STT-MRAM in 22FFL FinFET technology with 4ns read sensing time at 0.9 V using write-verify-write scheme and offset-cancellation sensing technique" IEEE International Solid-State Circuits Conference (ISSCC), 2019.